

ROVIO: Enclosure Sealing, Simulation, Control and Vision-Based Autonomy for an Underwater Vehicle

Joshua John Leo Valan Johnson

Roll No. 110123044

National Institute of Technology Tiruchirappalli
Robotics and Machine Intelligence (RMI) club



October 2026

Abstract

ROVIO is a student-built underwater vehicle of the Robotics and Machine Intelligence club at the National Institute of Technology Tiruchirappalli. This report presents my contributions to it: the watertight sealing of the enclosure, a simulation of the current eight-thruster vehicle, its closed-loop control, and the first components that take it from a remotely operated vehicle towards an autonomous one.

A flange seal with an EPDM gasket was selected after radial O-ring seals failed on an out-of-round acrylic tube, and it is used on the lid of the current enclosure. The vehicle was ported to the Stonefish simulator and placed in a coral reef produced by a purpose-written generator. A cascaded controller with thruster allocation holds depth, attitude, heading and position in simulation. Its gains were derived from a first-order model and corrected by identification of the simulated plant, whose effective inertia proved to be about seven times the assumed value; the roll loop was found to be bounded by a loop delay of about 90 ms and not by thrust. An extended Kalman filter supports depth and attitude hold on estimated state, while heading and horizontal position await an absolute reference.

For autonomy, the vehicle localizes itself from fiducial markers seen by its forward camera, with a mean position error of 0.05 m at 4 m when it faces a marker, and with a lateral uncertainty that is shown analytically to grow with the cube of the range. A perception pipeline detects corals, tracks them, estimates their positions from a single moving camera and maintains a map. On unattended survey flights the downward camera mapped 50 of the 53 corals that came into view with a mean position error of 0.20 m, both with ideal detections and with a detector trained on images that the simulator labelled automatically. On corals that the detector had never seen it mapped eleven of thirteen and assigned the correct family to nine.

Testing also exposed a fault in the simulator: its thrusters applied uninitialised setpoints during the first physics steps, which corrupted the vehicle state in 17 of 20 starts. The fault was traced to its source and corrected in the interface package.

All results from the simulation onwards were obtained in simulation with ground-truth vehicle state unless stated otherwise. Closing the loop on estimated state, and the transfer to the hardware, remain to be done.

Keywords: underwater robotics, remotely operated vehicle, Stonefish, thruster allocation, cascaded control, system identification, fiducial localization, monocular mapping, synthetic training data.

Contents

Nomenclature	6
1 Introduction	8
1.1 Context	8
1.2 Problem Statement and Objectives	8
1.3 Contributions	9
1.4 The Team and Authorship	10
1.5 Scope and Outline	10
2 Related Work	10
3 System Overview	11
3.1 Architecture	12
3.2 Frames and Conventions	12
4 Watertight Sealing of the Enclosure	13
4.1 Requirement	13
4.2 Seal Types Considered	13
4.3 Tests	13
4.4 Lessons for Printed Parts	15
4.5 Application to Version 3	15
5 Simulation	16
5.1 Choice of Simulator	16
5.2 Porting the Vehicle Model	16
5.3 Physical Model	17
5.4 Sensors	17
5.5 The Reef Scene	18
5.6 Generated Scenes	19
5.7 Light and Colour at Depth	20
5.8 Deployment	20
6 Control	21
6.1 Vehicle Model	21
6.2 Thruster Allocation	21
6.3 Cascaded Controller	23
6.4 Gain Design and Tuning	25
6.4.1 Design Rules	25
6.4.2 First Estimates	25
6.4.3 First Closed-Loop Tests	26
6.4.4 Identification of the Plant	26
6.4.5 Retuning	27
6.4.6 The Roll Loop	28
6.4.7 Result	29
6.5 Verification of Signs	30
6.6 Alternatives Considered	30
6.7 Teleoperation	31

7	State Estimation	31
7.1	Purpose and Status	31
7.2	Filter	31
7.3	Measurement Models	32
7.4	Observability	33
7.5	Integration with the Controller	33
7.6	Measurements	33
7.7	Fusion of the Marker Fixes	34
7.8	What Remains	35
8	Vision-Based Localization	35
8.1	Problem	35
8.2	Marker Map	35
8.3	Camera Model	35
8.4	Pose Estimation	36
8.5	Why a Single Marker Is Weak at Range	36
8.6	Covariance and Gating	38
8.7	Validation	38
9	Coral Perception and Mapping	40
9.1	Task	40
9.2	Detection	40
9.3	Automatic Labelling	40
9.4	Tracking	41
9.5	Range from a Single Camera	41
9.6	Map	43
9.7	Flights and Data	44
9.8	Scoring	44
9.9	Labelling Result	44
9.10	First Iteration	44
9.11	Second Iteration	45
10	Guidance	47
10.1	Waypoint Following	47
10.2	Survey Pattern	48
10.3	Safety	48
10.4	Result in Simulation	48
11	Verification and Faults Found	48
11.1	Approach	48
11.2	A Simulator Fault: Uninitialised Thruster Setpoints	49
11.3	Contact with Obstacles	50
11.4	Guards	51
12	Discussion and Limitations	51
12.1	Validity of the Results	51
12.2	Known Issues	51
12.3	Future Work	52
13	Conclusion	53

Acknowledgments	53
Appendices	57
A Chronology of the Project	57
B Deliverables	57

Nomenclature

Frames, vehicle state and control (Sections 3, 6, 7 and 10)

NED	world frame: x north, y east, z down
FRD	body frame: x forward, y starboard, z down
$\mathbf{p} = (x, y, z)$	position of the body origin in NED; z is depth
ϕ, θ, ψ	roll, pitch and yaw of the FRD body in NED (ZYX Euler angles)
$\mathbf{R}$	rotation matrix taking body coordinates to world coordinates
$\boldsymbol{\tau} \in \mathbb{R}^6$	body wrench: surge, sway and heave force, roll, pitch and yaw torque
$\mathbf{u} \in \mathbb{R}^8$	thruster forces (bold; the scalar u is the surge velocity)
κ	common scale factor applied to the thruster forces on saturation
$\mathbf{B} \in \mathbb{R}^{6 \times 8}$	thruster allocation matrix, $\boldsymbol{\tau} = \mathbf{B}\mathbf{u}$
$\boldsymbol{\eta}, \mathbf{v}$	pose in the world frame, velocity (u, v, w, p, q, r) in the body frame
$\mathbf{M}, \mathbf{D}, \mathbf{g}$	inertia with added mass, damping, restoring forces
$\mathbf{C}$	Coriolis and centripetal matrix
$\mathbf{S}$	rotation from the frame of the simulated model to FRD (Eq. (3))
m_i, d_i	effective inertia and linear drag of axis i
$k_o, v_{\max}$	gain and rate limit of the outer loop
T_o, T_i	time constants of the outer and the inner loop
$k_p, k_i, k_d, k_{\text{ff}}$	gains of the inner loop and its feed-forward
$l_{\max}, \tau_{\max}$	clamp of the integrator, limit of the inner output
F_b	feed-forward force that carries the net weight
$\tilde{\mathbf{a}}, \mathbf{a}$	specific force measured by the accelerometer, kinematic acceleration
P	gauge pressure
$k_g, v_{\max}$	gain and speed limit of the guidance
H	survey height above the seabed

Vision (Sections 8 and 9)

$\mathbf{K}, f$	camera intrinsic matrix, focal length in pixels
$\mathbf{X}_j, \mathbf{x}_j$	surveyed marker corner in the world, its observed pixel
$\boldsymbol{\xi}$	vehicle pose ($x, y, z, \phi, \theta, \psi$) estimated from the markers
$\pi(\cdot)$	pinhole projection of a world point for a given vehicle pose
$\sigma_{\text{px}}, \sigma_{\text{att}}$	standard deviation of corner detection and of the attitude prior
$\mathbf{o}_i, \mathbf{d}_i$	origin and unit direction of the i -th viewing ray to a coral
$\mathbf{c}, \mathbf{c}_L$	position estimate of a coral, position of a landmark of the map
s, r	side of a marker, range to a marker or a coral
$\sigma_{\perp}$	standard deviation of a marker fix across the line of sight
σ_{α}	angular standard deviation of a viewing ray
β	angle spanned by the viewing rays to a coral
n_{ik}	votes of coral i for object number k in the labelling
IoU	intersection over union of two bounding boxes
$\boldsymbol{\Sigma}$	covariance of an estimate

Abbreviations

ROV, AUV	remotely operated vehicle, autonomous underwater vehicle
DOF	degree of freedom
EKF	extended Kalman filter
IMU	inertial measurement unit
DVL	Doppler velocity log
PID	proportional, integral, derivative controller
ROS 2	Robot Operating System, version 2

1 Introduction

1.1 Context

Unmanned underwater vehicles fall into two classes. A remotely operated vehicle (ROV) is piloted by an operator through a tether. An autonomous underwater vehicle (AUV) carries out a mission on its own, using its sensors to determine where it is and what to do next. ROVIO, the Remotely Operated Underwater Vehicle for Inspection and Operation, is a project of the Robotics and Machine Intelligence (RMI) club of the National Institute of Technology Tiruchirappalli. Its long-term aim is a vehicle that can inspect and survey underwater sites without placing a diver at risk, and that is competitive in student competitions such as the Singapore AUV Challenge and RoboSub.

The project has passed through three versions.

1. **Version 1** was built by the previous team. It was an open-frame vehicle with six thrusters and five controllable degrees of freedom. Its frame of PVC pipe flexed under load, and its enclosure leaked.
2. **Version 2** was a deliberately simpler vehicle with four thrusters and four degrees of freedom (Fig. 1). I joined the project at this stage and worked on the watertight sealing of the enclosure.
3. **Version 3** is the current vehicle. It has eight thrusters and control authority in all six degrees of freedom (Fig. 2). For this version I moved to the newly formed simulation and control group.

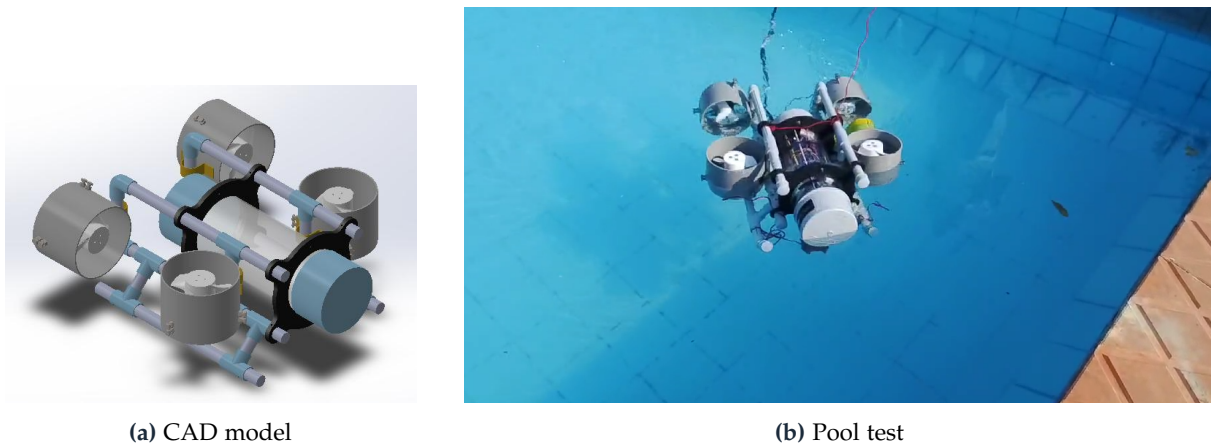


Figure 1. Version 2 of the vehicle: four thrusters on a PVC frame around an acrylic enclosure.

1.2 Problem Statement and Objectives

A student team cannot develop control and perception software on a vehicle that is still being machined, and it cannot afford to learn about sign errors and unstable loops in a pool. The work reported here therefore addresses two problems: keeping water out of the enclosure with the means of a student workshop, and providing a simulated vehicle on which the software of Version 3 can be developed, measured and tested before the hardware is ready. It had four objectives.

- To find a sealing method that keeps water out of the enclosure, can be opened and closed repeatedly, and fits the budget of a student club.

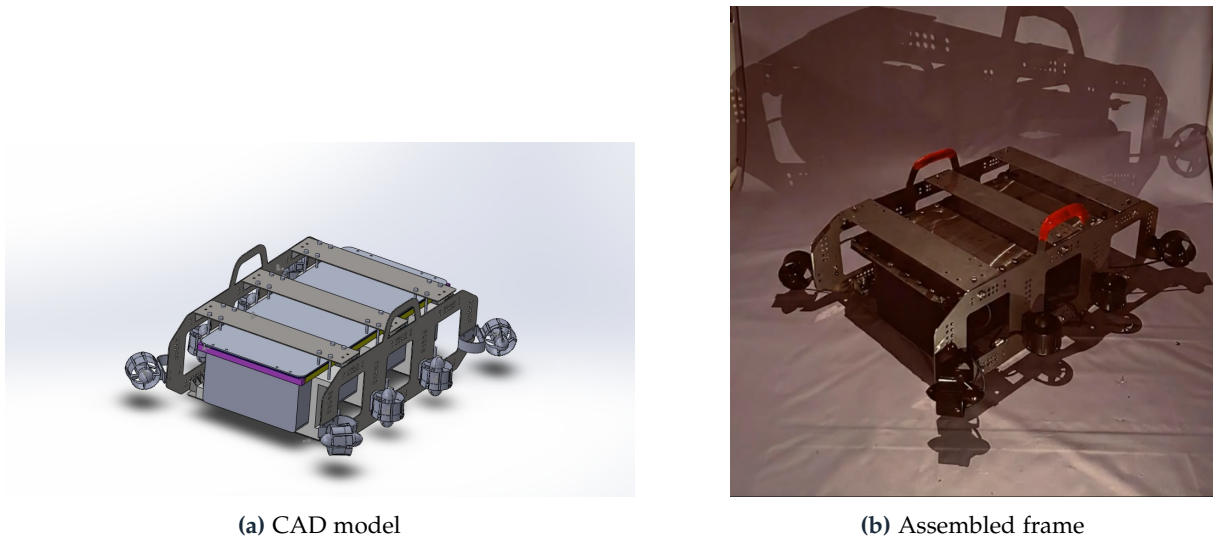


Figure 2. Version 3 of the vehicle: eight thrusters on a plate frame that carries rectangular enclosures.

- To build a simulation of the Version 3 vehicle that is faithful enough to develop control and perception software on.
- To give the simulated vehicle closed-loop control, so that it holds depth, attitude, heading and position when the pilot releases the controls.
- To take the first steps from a remotely operated vehicle towards an autonomous one: to let the vehicle localize itself from its camera, recognise and map what it sees, and fly a survey without a pilot.

1.3 Contributions

The contributions of this report are the following.

1. An experimental comparison of radial and flange seals for a low-pressure enclosure made with workshop tolerances, with an explanation of why the radial seal fails on an out-of-round bore and the flange seal does not (Section 4).
2. A Stonefish model of the eight-thruster vehicle and an open-source generator of coral reef scenes in which coral positions, marker positions and camera mountings each have a single source shared by the scene and the software (Section 5).
3. A cascaded controller with thruster allocation for six degrees of freedom, together with an identification of the simulated plant which shows that its effective inertia is about seven times the first estimate and that the roll loop is bounded by latency (Section 6).
4. An observability analysis and a first measurement of an inertial and pressure-based state estimator, which separates the states that can replace ground truth today from those that cannot (Section 7).
5. A marker-based localization with a verified covariance, and a closed-form result that the position uncertainty across the line of sight of a single marker grows with the cube of the range and is not reduced by an attitude prior (Section 8).
6. A monocular coral-mapping pipeline with automatic labelling of training data from the simulator, evaluated on held-out corals (Section 9), and a guidance layer that flies terrain-following surveys unattended (Section 10).

7. The diagnosis and correction of an uninitialised-memory fault in the simulator that corrupted the vehicle state in 17 of 20 starts (Section 11).

1.4 The Team and Authorship

ROVIO is the work of a team of the RMI club, organised in groups. The mechanical design of the vehicle is by Aditya Ganta, Arjun and Sreedath, with Arjun responsible for its water sealing. Saiprakash carried out procurement and manufacturing in Coimbatore, and Saiprakash and Srijan designed the printed circuit board of the vehicle. Kishore, Kazim, Sree Vidya and Nandgovind assisted in water sealing and assembly, and Krithyaa and Vignesh are further members of the team. The simulation group consists of Kuhan Anandh, Yagishvar and myself.

This report covers my own contributions, which fall into two parts. In the mechanical work I carried out the sealing study for Version 2 (Section 4). For Version 3 I identified the gasket material and helped to seal the enclosure; the design of that enclosure and its sealing as built are the work of the mechanical group.

The software described from Section 5 onwards is my own work: the port of the vehicle model and the scenes, the reef generator, the control stack, the state estimator, the marker localization, the perception pipeline, the guidance, and the tools and tests that accompany them. Three members of the team helped with it. Kuhan Anandh made the first Stonefish implementation in the team, which spawned a floating cube, and set up the repository from which this work started. Aditya Ganta helped with the conversion of the model from the east-north-up frame of the CAD export to the north-east-down frame of the simulator. Yagishvar advised on the teleoperation. The ROS 2 interface of the Stonefish simulator is open-source software by the author of the simulator, and I changed it only where this report says so (Section 11.2).

1.5 Scope and Outline

Everything from Section 5 onwards was developed and measured in simulation. No result in this report was obtained on the physical Version 3 vehicle, and where a number is quoted the text states how it was obtained. Most measurements are single runs; they establish that a function works and indicate its accuracy, and they do not support statistical claims (Section 12).

Section 2 places the work in the literature and Section 3 gives the architecture of the software and its conventions. Section 4 covers the sealing work. Section 5 describes the simulator, the vehicle model and the reef scene. Sections 6 and 7 describe control and state estimation. Sections 8 to 10 describe the three parts of the autonomy stack: localization from markers, coral perception, and survey guidance. Section 11 describes how the software was tested and the faults that testing exposed. Section 12 states the limitations of the results and what does not yet work, and Section 13 concludes. The chronology of the project and the list of deliverables are given in the appendices.

2 Related Work

This section places the design decisions of the report in the literature. It is selective: it covers the methods that were adopted and the alternatives against which they were chosen.

Simulation of underwater vehicles. Several open simulators serve marine robotics. UUV Simulator [1] extends Gazebo with hydrodynamic and thruster plugins that are parametrised by coefficients, and DAVE [2] builds on it with sensor models and environments for intervention tasks. HoloOcean [3] uses a game engine for rendering and offers sonar models of high fidelity. Stonefish [4] differs from the Gazebo-based tools in that it derives buoyancy, drag and added

mass from the geometry of the vehicle and renders the absorption and scattering of light in water. These two properties decided the choice made in Section 5: a team without tow-tank data cannot supply hydrodynamic coefficients, and a perception pipeline for corals needs images in which colour is lost with distance as it is in the sea [5]. The software is built on ROS 2 [6]. Frameworks such as Angler [7] address a different problem, the control of vehicles with manipulators.

Vehicle modelling and control. The model of a marine craft in six degrees of freedom, with added mass, damping and restoring forces, is standard [8, 9]. Controllers that exploit it, from model-based feed-forward to sliding-mode and model-predictive schemes, are surveyed in the same sources; all of them require coefficients that must be identified. The open autopilots used on small vehicles, PX4 [10] and its underwater relative ArduSub, instead use a cascade of a proportional pose loop around a PID rate loop, with the anti-windup and derivative-on-measurement provisions of standard PID practice [11]. This report adopts the cascade as a baseline for the same reason: it can be tuned without a model. Its known weakness, slow rejection of persistent disturbances, is commonly addressed by a disturbance observer [12], which is identified in Section 6.6 as the next step. The distribution of a commanded wrench over redundant thrusters is the control allocation problem, surveyed by Johansen and Fossen [13]; the unconstrained pseudo-inverse used here is its simplest solution, and the constrained formulations of that survey are the remedy for the saturation behaviour described in Section 6.2.

State estimation. Small underwater vehicles usually combine an inertial sensor, a pressure sensor and a Doppler velocity log in an extended Kalman filter [8, 9]. ROVIO carries no velocity log. The estimator of Section 7 uses the generic filter of the `robot_localization` package [14], and the section quantifies what the absence of a velocity measurement costs.

Localization from fiducial markers. Square fiducial markers such as AprilTag [15] and ArUco [16] give a camera a metric pose from four coplanar corners, and they have been evaluated under water, where turbidity and attenuation shorten the range at which they are detected [17]. The pose of a planar target is known to be ambiguous: two orientations explain the observed corners almost equally well when the target is small in the image [18, 19]. Solvers return both candidates and leave the choice to the application. The localization of Section 8 resolves the ambiguity with the attitude from the inertial sensor, entered as a weighted prior in a nonlinear least-squares problem solved by the Levenberg-Marquardt method [20, 21], and the section derives how far such a prior can help.

Object detection, tracking and monocular mapping. Single-stage detectors of the YOLO family [22, 23] are the usual choice for real-time detection on embedded computers. Tracking by detection associates boxes between frames by their overlap, solved as an assignment problem [24] as in SORT [25]; ByteTrack [26] adds a second association pass for low-confidence detections, which is the scheme adopted in Section 9. A single camera observes bearings only, and the position of a landmark follows from triangulation over a baseline [27], with an uncertainty along the viewing direction that depends strongly on the parallax, as in monocular SLAM [28]. Training detectors on simulated images with randomised appearance is an established way of reducing the dependence on the simulator [29]; the augmentation of Section 9.3 applies it with perturbations chosen to resemble the effect of water on an image [5].

3 System Overview

3.1 Architecture

The software is a set of ROS 2 packages that communicate only through topics (Fig. 3). The simulator, with its ROS 2 interface, stands in for the vehicle: it accepts eight thruster forces and publishes the sensor data and, in simulation only, the true state. Control is divided into a controller, which turns velocity commands and the vehicle state into a body wrench, and an allocator, which turns the wrench into thruster forces. The marker localization and the coral perception consume camera images and publish estimates with covariances; neither commands the vehicle. The guidance converts a route into velocity commands on the topic that teleoperation also uses, so that an autonomous flight and a piloted one are indistinguishable to the controller.

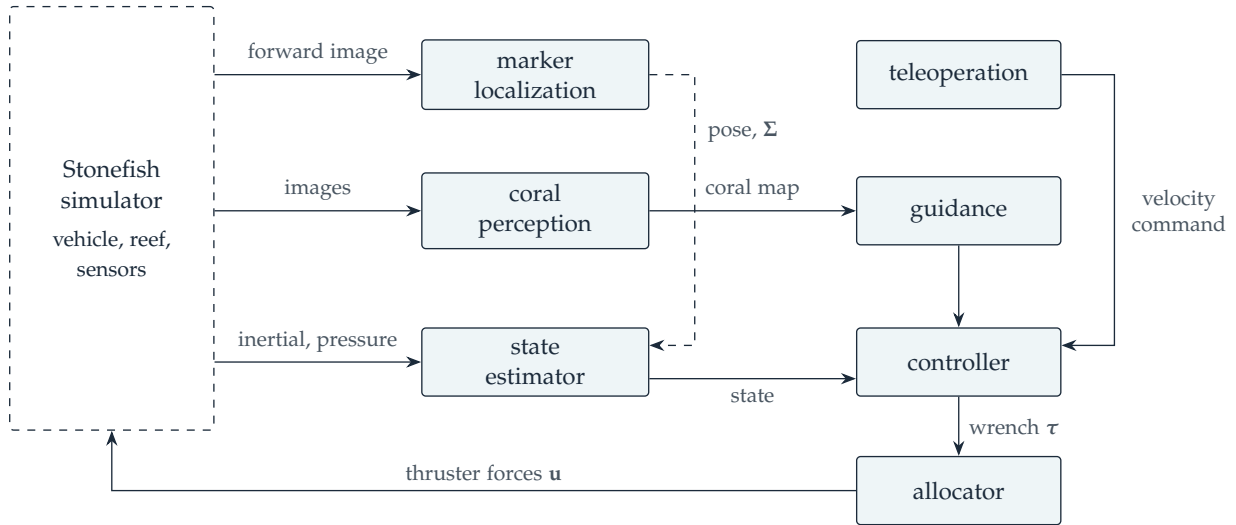


Figure 3. Architecture of the software. Each box is a ROS 2 package and each arrow a topic. Teleoperation and guidance publish to the same command topic and are alternatives, not a chain. In the reported results the state is the ground truth of the simulator; the estimator and its marker input (dashed) are the path that replaces it.

Three properties of this structure recur in the report. First, each quantity that is needed in two places is defined in one: the coral positions, the marker map and the camera mountings are each written in a single file from which both the simulator scene and the software are generated (Section 5). Second, the arithmetic of every package is kept in modules that do not depend on ROS 2, so that it can be tested without the simulator (Section 11). Third, the interface between autonomy and control is a velocity command with a timeout, on which the controller holds its pose when the command stops; every failure of a higher layer therefore ends in a hold (Sections 6.3 and 10).

3.2 Frames and Conventions

The software uses the conventions of marine vehicle dynamics [8]: a north-east-down (NED) world frame and a forward-starboard-down (FRD) body frame. Depth is positive downwards, positive roll lowers the starboard side, positive pitch raises the nose and positive yaw is clockwise seen from above. The attitude is described by ZYX Euler angles, which are obtained from the rotation matrix $\mathbf{R}$ (body to world) as

$$\phi = \text{atan2}(R_{32}, R_{33}), \quad \theta = -\arcsin R_{31}, \quad \psi = \text{atan2}(R_{21}, R_{11}). \quad (1)$$

The body frame of the simulated model is different: because of the way the CAD model was exported, its x axis points forward, its y axis up and its z axis to starboard (Section 5.2).

The FRD axes written in the coordinates of that frame are the columns of

$$\mathbf{S} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}, \quad (2)$$

and the quantities reported by the simulator are converted by

$$\mathbf{R} = \mathbf{R}_{\text{sim}} \mathbf{S}, \quad \mathbf{v} = \mathbf{S}^\top \mathbf{v}_{\text{sim}}, \quad \boldsymbol{\omega} = \mathbf{S}^\top \boldsymbol{\omega}_{\text{sim}}. \quad (3)$$

Equation (3) is implemented in a single function, and no other part of the software knows that the simulated body is *y*-up. On the real vehicle the function is replaced by the mounting rotation of the inertial sensor. The control software was first written in the frame of the model and was moved to these conventions on 23 September 2026. The rank and conditioning of the allocation matrix and the gains were unaffected, since the change is a pure rotation.

4 Watertight Sealing of the Enclosure

4.1 Requirement

Version 1 suffered from persistent leaks. For Version 2 the requirement was a seal that is static (it sits between two parts that do not move relative to each other), temporary (the enclosure must be opened to reach the electronics), and tight at the depth of the test pool, about 3 m of water. The hydrostatic pressure at that depth,

$$p = \rho gh \approx 1000 \text{ kg/m}^3 \times 9.81 \text{ m/s}^2 \times 3 \text{ m} \approx 29 \text{ kPa}, \quad (4)$$

is less than a third of an atmosphere. This is small by the standards of sealing practice, and it shaped the problem: the difficulty was not the strength of the seal but obtaining a continuous line of contact between parts that a student workshop can make.

4.2 Seal Types Considered

Three kinds of static seal were examined.

Radial seal. An O-ring sits in a groove around a cylindrical plug and is compressed against the bore of the tube. This is the arrangement used on a piston. Its performance depends on the cross-section and inner diameter of the ring, on the groove dimensions and on how round the bore is.

Face seal. An O-ring is compressed axially between two flat faces.

Flange seal. A flat gasket covers a broad annular area between two flanges that are clamped by a ring of bolts. This is the arrangement used on the vehicles of several established university teams.

4.3 Tests

The radial seal was tested first, because it needs the fewest parts. Test plugs were printed in PLA so that groove dimensions could be changed quickly (Fig. 4). Discs carrying a single O-ring were printed in a series of diameters to find the compression at which the ring sealed without making the plug impossible to insert.

The radial seal did not become fully tight. An O-ring seals by being squeezed between the groove and the opposing wall, and it does so only where the squeeze is positive all the way

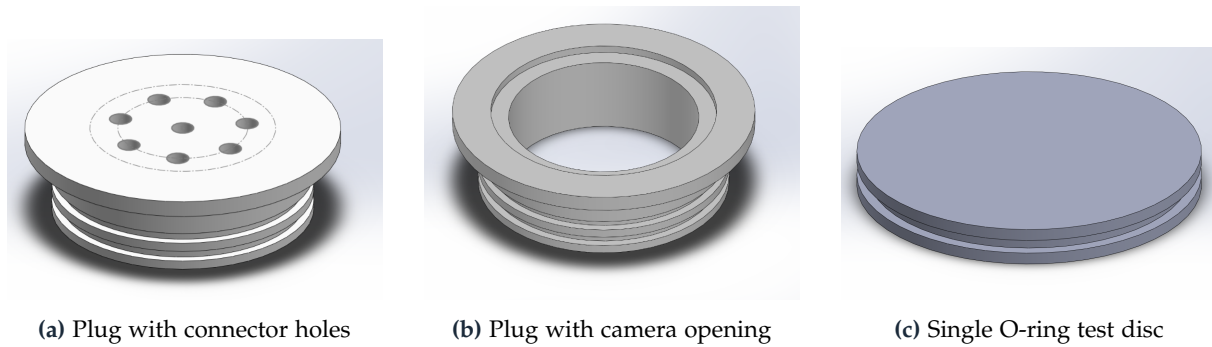


Figure 4. Printed test pieces for the radial seal.

round. The gap that the ring has to close is the difference between the bore of the tube and the diameter of the plug, and on a bore that is not round this gap varies around the circumference. A plug large enough to squeeze the ring at the widest part of the bore no longer enters at the narrowest. The acrylic tube was of this kind: a small leak remained even at the largest plug diameter that the tube would accept. A low pressure does not help here, because an O-ring relies on the pressure itself to press it against the downstream wall of its groove, and 29 kPa contributes little.

As a stopgap for the pool tests of Version 2, the tube was closed with standard PVC end caps sealed with PTFE tape and paraffin wax. This was tight, because both mating surfaces are manufactured to tolerance, but the caps were difficult to remove, so it does not meet the requirement of a seal that can be opened routinely.

Two further ideas were tried as additions to the O-ring. Silicone grease on the ring improved the seal slightly by filling surface roughness. Filling the groove with oil was considered and rejected, because releasing oil into open water is not acceptable.

The flange seal was then tested on a dedicated rig (Fig. 5). It sealed reliably, and the reason is the converse of the failure above. A flat gasket is compressed along the axis of the enclosure by a ring of bolts, so the contact pressure is set by the bolts and not by the fit of two diameters, and the roundness of the tube no longer enters. The gasket also covers a broad annular area, so a local defect in a face does not open a path across the whole seal. The parameters that mattered in the tests were the width of the sealing area, the depth of the groove that locates the gasket, which determines how far it is compressed before the faces meet, and the hardness of the gasket.

The gasket material was chosen on the last of these. EPDM rubber has a lower Shore hardness and is more compressible than the nitrile rubber of standard O-rings, and it therefore conforms to a printed or machined face at a lower bolt load.

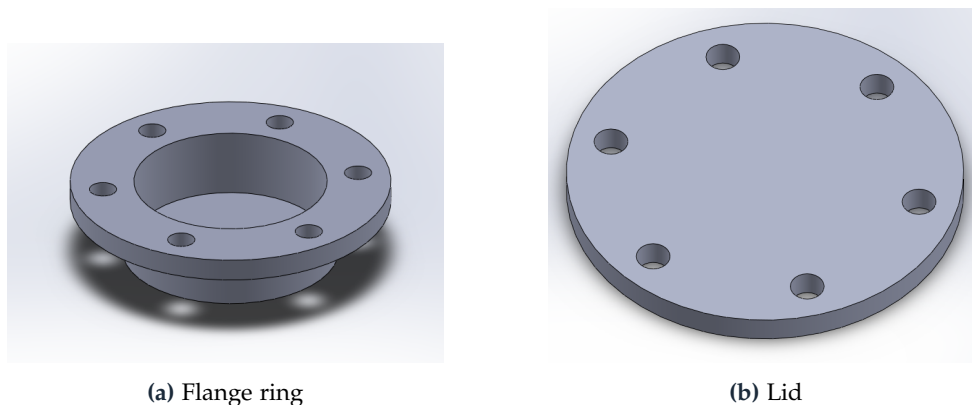


Figure 5. Test rig for the flange seal.

4.4 Lessons for Printed Parts

The test pieces also showed how parts printed in PLA behave under water. A printed wall is not a continuous solid: water passes between the layers and along the seams of a part that looks closed. Printed parts that have to hold water out must therefore be finished after printing so that their surface is closed, and a sealing face has to be smooth across the layer lines before a gasket or a ring can seal against it. The same finishing is applied to the printed parts of Version 3.

4.5 Application to Version 3

Version 3 uses a rectangular enclosure whose top is closed by a flanged lid with a ring of bolts (Fig. 6). The lid is sealed with a flange seal of the kind studied above, which gives access to the electronics from the top without disturbing the frame. My part in this version was limited: I identified EPDM as the suitable gasket material and helped to seal the enclosure. The enclosure and its sealing were designed by the mechanical group, in which Arjun was responsible for the water sealing, and Kishore, Kazim, Sree Vidya and Nandgovind assisted in sealing and assembly. Before the arrangement was adopted it was tested in hardware on a stainless steel box of the same form, closed by a bolted lid and held under water in a tub (Fig. 7).

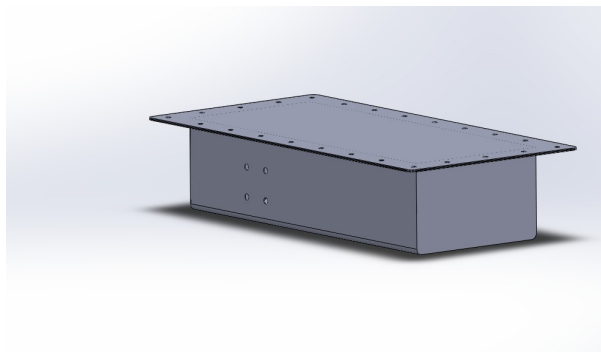


Figure 6. Enclosure of Version 3 with the flange for the sealed lid.

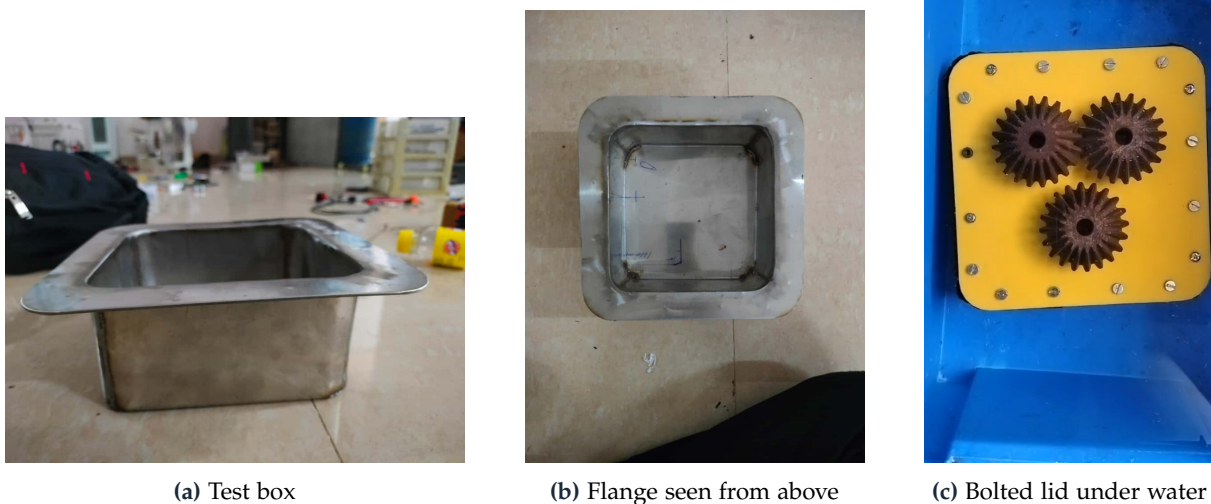


Figure 7. Hardware test of the flange seal before its use on Version 3.

I was part of the mechanical design group for Version 2 only. The mechanical design of Version 3 is the work of Aditya Ganta, Arjun and Sreedath, and the remainder of this report

concerns the simulation group, to which I moved because the project needed someone with a background in control and in ROS 2.

5 Simulation

5.1 Choice of Simulator

Before the Version 3 model was available, a REMUS vehicle model was simulated in Gazebo to learn how drag and buoyancy are configured (Fig. 8). The exercise showed that the stock Gazebo buoyancy and hydrodynamics plugins are laborious to configure and that the simulation was unstable, so three alternatives were compared.

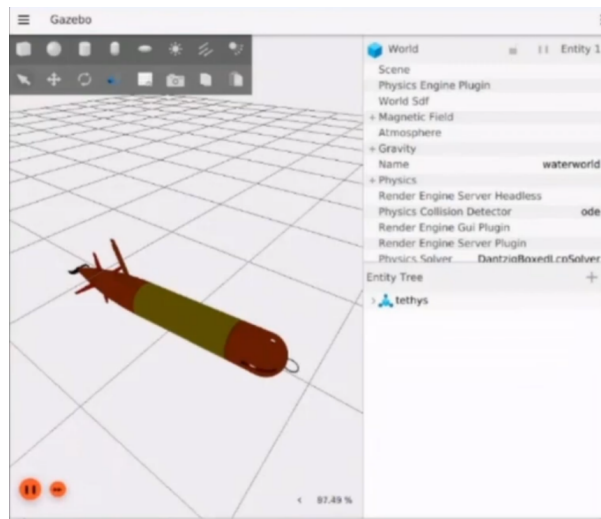


Figure 8. REMUS vehicle model in Gazebo, used for a first study of drag and buoyancy.

Isaac Sim with OceanSim. It offers the most realistic rendering and integrates with ROS 2, but it needs more computing power than the club has and takes long to learn.

Stonefish. It is a simulator written for marine robotics [4]. It computes buoyancy and drag from the geometry of the vehicle, renders underwater light absorption and scattering, and provides the sensors needed here. It runs on a laptop graphics card. Its physics is built on the Bullet engine.

Angler. It is not a simulator but a framework for vehicles with manipulators that are controlled through ArduSub and simulated in Gazebo [7].

Stonefish was chosen. Angler was set aside because the vehicle has no manipulator and its controller does not run on ArduSub in simulation. For the same reason a ground control station such as QGroundControl has no role in the simulated system; it becomes useful only if the hardware is flown on an ArduSub autopilot.

The first Stonefish set-up in the team was made by Kuhan Anandh in December 2025. It spawned a floating cube, and it showed the simulator and its ROS 2 interface running before a vehicle model existed. The work described below started from it.

5.2 Porting the Vehicle Model

The CAD model was exported from SolidWorks as a URDF description and converted to the Stonefish scenario format. Three problems had to be solved.

Frames. Stonefish uses a north-east-down world frame, whereas the URDF exporter assumes an east-north-up frame, and the Stonefish documentation does not state this prominently. A model imported as exported is wrongly oriented and its thrusters are misaligned. Every quantity of the description therefore had to be re-expressed. For a change of axes given by an orthogonal matrix $\mathbf{T}$, positions and directions transform as vectors and the inertia as a tensor,

$$\mathbf{r}' = \mathbf{T}\mathbf{r}, \quad \mathbf{I}' = \mathbf{T}\mathbf{I}\mathbf{T}^\top, \quad (5)$$

which was applied to the link poses, the centre of mass and the inertia of the hull and of each thruster. Aditya Ganta helped with this conversion. The meshes of the thrusters, which carry their own axes, were re-oriented in Blender. The body frame that results has x forward, y up and z to starboard, and the scenario stands the vehicle upright with a roll of -90° . This body frame is confined to one function of the control software (Section 3.2).

Thruster directions. With the frames corrected, the directions of the thrusters in the model still did not agree with those assumed by the allocation, and after they had been corrected a residual torque remained, which was compensated in February 2026. These faults were found by driving the vehicle; the test of Table 14 was written later so that they would be found without driving it.

Mesh size. The exported meshes contained more than 600 000 triangles, which slowed the simulation. They were decimated in MeshLab; the hull mesh now has 30 000 triangles.

5.3 Physical Model

Stonefish computes the hydrostatic and hydrodynamic forces from the geometry of the body, so they do not have to be entered as coefficients. Buoyancy follows from the submerged volume of the mesh, and drag and added mass from simple shapes fitted to it. Table 1 gives the parameters of the model. The thrusters are of the simplest kind the simulator offers: each applies the commanded force along its axis at once, without rotor dynamics, dead band or the asymmetry between forward and reverse thrust that real thrusters have. The water is still, with no current and no waves. These simplifications are appropriate for developing the software and are among the differences from the hardware that have to be closed later (Section 12.2).

Quantity	Value
Mass	14.73 kg
Inertia about the roll, pitch and yaw axes	0.149, 0.220, 0.314 kg m ²
Weight, net weight in water	145 N, 75 N (measured, Table 9)
Thrusters	8, each ± 100 N, ideal force sources
Water density	1031 kg/m ³
Physics rate	100 Hz

Table 1. Parameters of the simulated vehicle. Mass, inertia and thruster limits are the values of the scene file and have not been measured on the hardware.

5.4 Sensors

Table 2 lists the simulated sensors. The downward camera and the two segmentation cameras were added during the perception work, and the forward camera was pitched down then (Section 9.11). A segmentation camera renders, for every pixel, the identity of the object seen there. It exists only in simulation and is used to produce training labels and as a ground-truth detector (Section 9).

Sensor	Rate	Notes
Inertial measurement unit	50 Hz	orientation, angular rate, acceleration
Pressure sensor	50 Hz	gauge pressure, converted to depth
Forward camera	15 Hz	1280 × 720 pixels, 80° field of view, pitched 20° down
Downward camera	15 Hz	same optics, on the hull underside, image top towards the bow
Segmentation cameras	15 Hz	one beside each colour camera, simulation only
GPS	1 Hz	usable at the surface only, not used
Ground-truth odometry	50 Hz	simulation only, used as the reference in all tests

Table 2. Sensors of the simulated vehicle. Camera rates are nominal; the rate delivered near the reef is lower (Section 5.5).

The position and orientation of each camera on the body are needed in two places: in the scene, where the simulator renders from them, and in the software, which projects world points into the image with them. An error of a few degrees between the two would bias every result of Sections 8 and 9 without any visible symptom. The mountings are therefore defined once, in a file of the localization package. The scenes are generated from that file and the software imports it. The direction of the downward camera was verified in the simulator by motion: with the vehicle moving forward the image content must move towards the bottom of the frame, and with the vehicle moving to starboard it must move to the left.

5.5 The Reef Scene

A scene was needed in which the vehicle has something to look at and to collide with. I wrote a generator that assembles a coral reef from a layout file and a set of coral models. It is kept in a repository of its own, published under an open licence, and is included in the project as a submodule, so that it can be used with any Stonefish scenario and not only with this vehicle.

Content. The seabed is a height field of 60 m × 60 m on a grid of 0.25 m, between 11.7 and 16.7 m deep, with a mound whose top lies at (2, −3) m. The layout places 111 corals on it: 41 densely on the mound, within 10 m of its top, and 70 scattered over the surrounding sand. They are instances of 33 models of nine families (Table 3), which were generated procedurally from reference images of real species. Each instance has its own position, heading and scale.

Family	Models	Instances	Largest width	Height
Anemone	1	3	1.1 m	0.46 m
Antler	5	21	0.6 m	0.31 to 0.39 m
Blue	2	8	0.9 m	0.38 to 0.62 m
Bush	5	14	3.1 m	0.31 to 1.41 m
Clathrata	3	23	0.6 m	0.10 to 0.12 m
Plate	8	16	4.3 m	0.14 to 0.47 m
Soft	2	6	0.9 m	0.66 to 1.24 m
Staghorn	4	16	2.1 m	0.38 to 1.17 m
Table	3	4	5.1 m	1.35 to 2.10 m
All	33	111		

Table 3. Coral families of the reef layout, with the sizes of their models before the scale of the instance is applied. The families are the classes of the detector of Section 9.

Single source. The layout file is the only place in which a coral position is written. The generator produces the scenario from it, either as a complete scene or merged into an existing one, in which case the environment, vehicle, sensors and materials of the host scene are kept. The meshes are authored with z up and their origin at the base of the coral; a point (x, y, z) of a mesh appears in the NED world at $(x, -y, 15 - z)$. The perception software reads the coral positions back from the generated scenario and uses them as ground truth (Section 9.8).

Number of corals. The number of corals is an option. A number below 111 keeps the largest ones. A number above 111 adds corals at seeded random positions: model and scale are drawn with the frequencies and ranges of the layout, the split between mound and sand is preserved, and a candidate is rejected when it would stand closer to another coral than 0.9 times the sum of their footprint radii, within 0.6 m of a rock or within 1 m of the border. The scene went through two sizes. The measurements of Section 9 were made in a reef of 70 corals. The reef was then enlarged to 150, of which 70 are those of the earlier scene, unchanged, and the earlier scene is kept so that the recorded flights can be scored against it.

Collision meshes and level of detail. The models have 14 000 to 40 000 triangles each. For collision every model has a reduced mesh of about 3 000 triangles, made in Blender by edge collapse. Models that consist of dense fields of thin blades, or whose meshes are not closed, are first converted to a voxel volume, since collapsing them directly leaves loose shards; the collision mesh of such a model is a solid envelope. With the full models the reef of 111 corals has 4.2 million triangles to display. An option of the generator uses the collision meshes for display as well, which reduces this to 0.47 million, and the project uses it, because rendering limits the simulation near the reef. On a laptop graphics card (RTX 3060) the simulator delivers about five camera images per second in total at the lowest quality setting, shared between the cameras, with 150 corals in the scene, and about seven with 70. The physics is not affected and runs at its nominal rate.

5.6 Generated Scenes

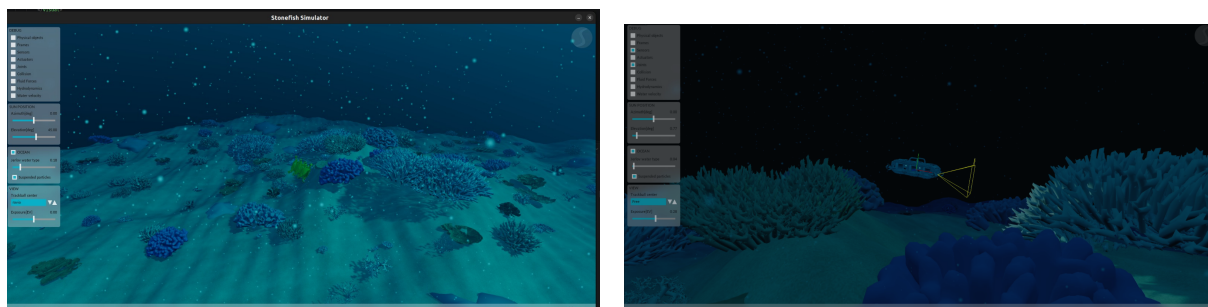
The scenes of the project are produced in three steps, none of which is edited by hand afterwards: the base scene with the vehicle in open water; the reef, merged into it by the generator above; and a second generator that adds what the autonomy stack needs. The second generator writes the marker boards of Section 8 from the marker map, the downward camera and both lamps from the camera mountings, and the start pose of the vehicle; for the scene used in recording it also adds the segmentation cameras and sets the noise of the inertial sensor to zero. Running it twice changes nothing. Table 4 lists the results.

Scene	Content
Open water	vehicle, boards at their reef positions, no seabed
Plane	vehicle and boards above a flat seabed
Reef (default)	reef of 150 corals, boards, start pose beside the mound
Reef, vision	as the default, with segmentation cameras and a noise-free inertial sensor
Reef, vision, 70 corals	the earlier reef, kept for the recorded flights

Table 4. Scenes generated for the project.

The start pose is checked when it is written. The generator loads the collision mesh of every object, transforms it into the world and measures the distance from the start position to the nearest vertex; a position closer than 1 m is refused. The check was added after it was

found that a vehicle whose hull overlaps a collision mesh at start is thrown out of the scene (Section 11.3), and it replaced an earlier check against bounding circles that had given a wrong answer (Section 11.2).



(a) Overview

(b) Vehicle above the reef

Figure 9. The coral reef scene in Stonefish, before the coral colours were changed (Section 5.7).

5.7 Light and Colour at Depth

In the first runs near the seabed every coral appeared grey-green, although each model had been given a distinct colour. The cause was not the models. Stonefish attenuates light along its path through the water, and red is absorbed first [5]. Sunlight that has travelled 12 m down contains almost no red, and the light of a lamp on the vehicle loses its red again on the way to the coral and back. This is physically correct and is the reason divers carry lamps and still see little colour beyond a few metres.

Three changes were made so that the corals keep a recognisable hue in the camera image (Fig. 10).

- The forward lamp of the vehicle was made ten times brighter and inclined downwards, and a second lamp was added beside the downward camera.
- The reef generator received an option that writes saturated colours whose red component is amplified above the nominal maximum: the saturation of each model colour is raised by a factor of 1.8, its brightness to at least 0.65, and the red, green and blue components are then multiplied by 5, 1.5 and 1. A red gain of five compensates for the absorption over a light path of a few metres.
- The vehicle now starts 1.5 m above the sand beside the reef mound instead of at the surface, at a position that the scene generator checks against every collision mesh (Section 5.6).

Colour is still lost at range: a coral 3 m away shows its hue, one 8 m away does not. This is retained deliberately, because a detector trained on such images has to cope with the same effect in the sea.

5.8 Deployment

The simulator, ROS 2 Humble [6] and all dependencies are packaged in a Docker image, so that every member of the team runs the same environment. The source tree is mounted into the container and rebuilt at start, which allows code to be edited on the host without rebuilding the image. A second image variant adds the neural network library needed for training.

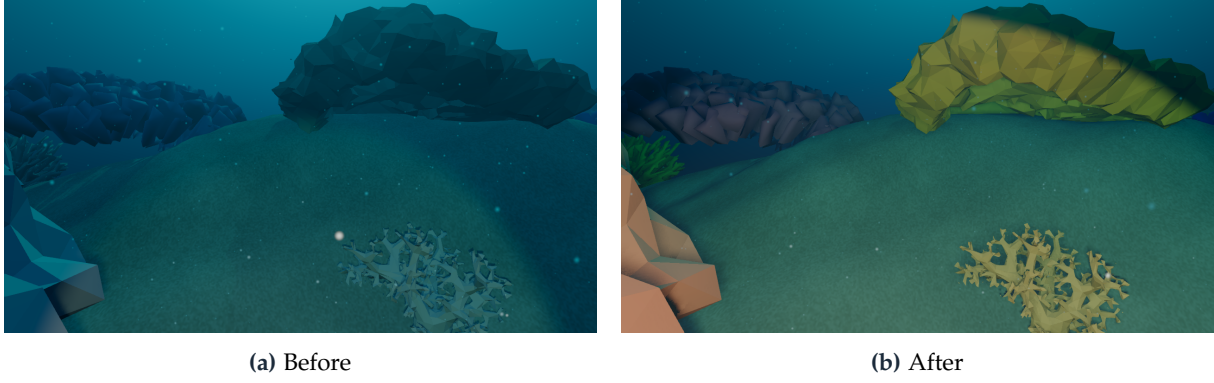


Figure 10. Forward camera at 12 m depth from the same position, before and after the changes to the lamps and the coral colours. The camera was still level when these images were taken.

6 Control

This section describes the control stack in the order in which a command passes through it backwards: the vehicle model, the allocation of a wrench to the thrusters, the cascaded controller that produces the wrench, and the design, identification and tuning of its gains. The frames are those of Section 3.2.

6.1 Vehicle Model

The motion of an underwater vehicle is described by [8]

$$\dot{\eta} = \mathbf{J}(\eta) \mathbf{v}, \quad \mathbf{M}\dot{\mathbf{v}} + \mathbf{C}(\mathbf{v})\mathbf{v} + \mathbf{D}(\mathbf{v})\mathbf{v} + \mathbf{g}(\eta) = \boldsymbol{\tau} + \boldsymbol{\tau}_d, \quad (6)$$

where $\eta = (x, y, z, \phi, \theta, \psi)$ is the pose in the world frame, $\mathbf{v} = (u, v, w, p, q, r)$ the velocity in the body frame, $\mathbf{M}$ the inertia including the added mass of the water that the vehicle accelerates with it, $\mathbf{C}$ the Coriolis and centripetal terms, $\mathbf{D}$ the damping, $\mathbf{g}$ the restoring forces of weight and buoyancy, $\boldsymbol{\tau}$ the wrench of the thrusters and $\boldsymbol{\tau}_d$ the disturbances (current, tether).

The controller of this report does not contain this model. It uses two of its terms in the simplest possible form: the constant part of $\mathbf{g}$, the net weight of the vehicle, as a feed-forward force, and the slope of $\mathbf{D}$ on each axis as a feed-forward gain. Each axis is otherwise treated as an independent first-order system,

$$m_i \dot{v}_i + d_i v_i = \tau_i + b_i, \quad (7)$$

with an effective inertia m_i , a linear drag d_i and a slowly varying bias b_i that the integrator removes. The Coriolis terms are neglected, which is justified at the speeds flown (below 0.5 m/s), and the coupling between axes is left to the feedback. Section 6.4 gives the values of d_i measured in the simulator and shows where the assumption of independent axes fails.

6.2 Thruster Allocation

The vehicle has eight thrusters for six degrees of freedom, so it is over-actuated, and the distribution of a wrench over them is a control allocation problem [13]. With thruster i at position $\mathbf{r}_i$ and thrusting along the unit vector $\mathbf{e}_i$, both in the body frame, the wrench produced by the thruster forces $\mathbf{u}$ is

$$\boldsymbol{\tau} = \mathbf{B} \mathbf{u}, \quad \mathbf{B} = \begin{bmatrix} \mathbf{e}_1 & \cdots & \mathbf{e}_8 \\ \mathbf{r}_1 \times \mathbf{e}_1 & \cdots & \mathbf{r}_8 \times \mathbf{e}_8 \end{bmatrix}. \quad (8)$$

Thrusters 1, 2, 7 and 8 stand at the four corners in the horizontal plane, at 45° to the longitudinal axis, and produce surge, sway and yaw. Thrusters 3 to 6 are vertical and produce heave, roll and pitch. With the positions of the model, in metres,

$$\mathbf{B} = \begin{bmatrix} 0.707 & 0.707 & 0 & 0 & 0 & 0 & -0.707 & -0.707 \\ -0.707 & 0.707 & 0 & 0 & 0 & 0 & -0.707 & 0.707 \\ 0 & 0 & -1 & -1 & -1 & -1 & 0 & 0 \\ 0.010 & -0.010 & -0.247 & 0.247 & -0.247 & 0.247 & 0.010 & -0.010 \\ 0.010 & 0.010 & 0.102 & 0.102 & -0.102 & -0.102 & -0.010 & -0.010 \\ -0.425 & 0.425 & 0 & 0 & 0 & 0 & 0.425 & -0.425 \end{bmatrix}. \quad (9)$$

The small entries in the roll and pitch rows of the horizontal thrusters arise because these thrusters sit 14 mm below the origin of the body frame. Figure 11 shows the arrangement that this matrix describes.

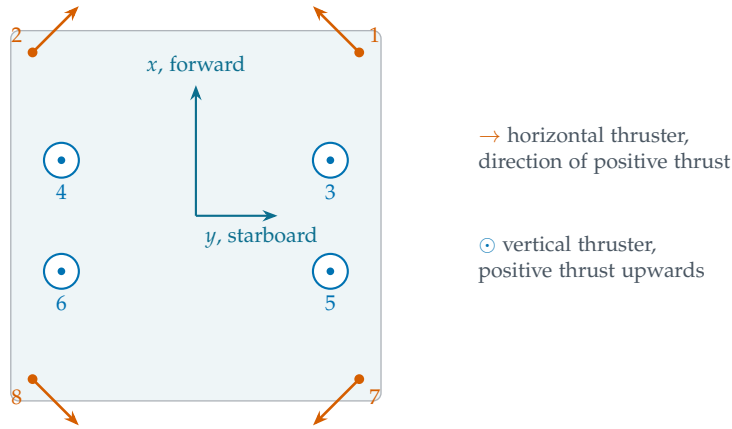


Figure 11. Thruster arrangement seen from above, reconstructed from $\mathbf{B}$. The vertical thrusters are drawn at their positions, 0.247 m to either side of the centre and 0.102 m ahead of and behind it; their short longitudinal arm is the reason pitch is the weakest axis. The positions of the horizontal thrusters are indicative: only their directions and their yaw arm of 0.425 m follow from the matrix.

Controllability. $\mathbf{B}$ has rank 6, so every wrench can be produced. Its singular values are 2.00, 1.41, 1.41, 0.85, 0.49 and 0.20, and its condition number is 9.76. Because the two groups of thrusters are almost decoupled, each singular value belongs to one axis: heave (four parallel thrusters, $\sqrt{4} = 2$), surge and sway ($\sqrt{4} \cdot 0.5 = 1.41$), yaw ($2 \cdot 0.425$), roll ($2 \cdot 0.247$) and pitch ($2 \cdot 0.102$). Pitch is the weakest axis by a factor of two and a half, because the vertical thrusters stand only 0.10 m ahead of and behind the centre while they stand 0.25 m to either side of it. Table 5 gives the largest wrench available on each axis alone.

	Surge	Sway	Heave	Roll	Pitch	Yaw
Largest wrench	255 N	255 N	360 N	89 N m	37 N m	153 N m

Table 5. Largest wrench on one axis alone at the thruster limit of 90 N, computed from the pseudo-inverse of $\mathbf{B}$.

Inversion. Since $\mathbf{B}$ has more columns than rows, Eq. (8) has infinitely many solutions for a given wrench. The allocator uses the Moore-Penrose pseudo-inverse,

$$\mathbf{u} = \mathbf{B}^+ \boldsymbol{\tau}, \quad \mathbf{B}^+ = \mathbf{B}^\top (\mathbf{B} \mathbf{B}^\top)^{-1}, \quad (10)$$

which is the solution of least $\|\mathbf{u}\|_2$, that is, of least total squared thrust. The two remaining degrees of freedom, combinations of thrust that push against each other without moving the vehicle, are thereby left at zero. A weighted form is implemented as well, so that a weak or failed thruster can be given a lower share, but it is not used at present.

Saturation. The pseudo-inverse ignores the thruster limit. When the largest command exceeds $u_{\max} = 90 \text{ N}$ (the model allows 100 N ; the difference is a margin), all eight commands are scaled by the same factor,

$$\mathbf{u} \leftarrow \kappa \mathbf{u}, \quad \kappa = \min\left(1, \frac{u_{\max}}{\max_i |u_i|}\right), \quad (11)$$

so that the delivered wrench is $\kappa \boldsymbol{\tau}$. The direction of the wrench is preserved, and a saturated vehicle moves more slowly in the requested direction instead of veering off. The price is that every axis is reduced together: a heave demand that saturates the vertical thrusters also reduces the roll and pitch torque that the same thrusters were asked for. The controller guards against this by limiting the output of each axis to a share of the budget (Section 6.4). An allocation by quadratic programming, which would respect the limits and give attitude priority [13], is the proper remedy and is left as future work.

The first teleoperation scheme connected the joystick to this allocator directly. The vehicle could be driven in all six degrees of freedom, but it drifted as soon as the sticks were released, because nothing opposed its weight or residual motion.

6.3 Cascaded Controller

Closed-loop control was added so that the vehicle behaves like a multirotor: it moves while a stick is deflected and holds its pose when the stick is released. Each degree of freedom has a cascade of two loops (Fig. 12), the structure used by the PX4 and ArduSub autopilots, which was chosen so that the experience of the hardware group with ArduSub carries over [10].

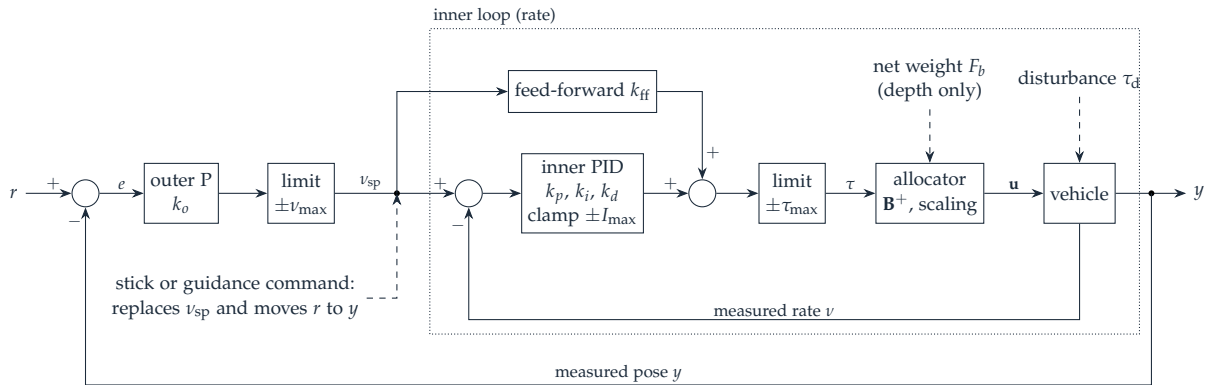


Figure 12. Control cascade of one degree of freedom, with the symbols of Eqs. (12) and (13). The outer loop turns the pose error into a limited rate setpoint; the inner loop, closed on the measured rate, turns the rate error into a force or torque, to which the drag feed-forward is added. The allocator converts the wrench of all six axes into thruster forces. A non-zero command bypasses the outer loop.

For a pose error e and a measured rate v , the outer loop produces a rate setpoint and the inner loop a force or torque:

$$v_{\text{sp}} = \text{sat}_{v_{\max}}(k_o e), \quad (12)$$

$$\tau = \text{sat}_{\tau_{\max}}\left(k_p(v_{\text{sp}} - v) + k_i \int (v_{\text{sp}} - v) dt - k_d \dot{v} + k_{\text{ff}} v_{\text{sp}}\right). \quad (13)$$

The integral is clamped to $\pm I_{\max}$ against wind-up, and the derivative acts on the measurement and not on the error, so that a step in the setpoint does not produce a kick; both are standard provisions [11]. The feed-forward term supplies the drag expected at the commanded rate, $d_i v_{\text{sp}}$ in Eq. (7), so that the feedback terms only correct the remainder; it lies outside the loop and cannot destabilise it.

The outer loop is proportional only. The inner integrator already removes steady disturbances, and the inner loop, acting on the derivative of the pose, already supplies the damping; integral and derivative action in the outer loop would add wind-up and slow oscillation without a benefit. With an inner loop that tracks well, the pose obeys $\dot{e} = -k_o e$, so $1/k_o$ is the time constant of the outer loop.

World and body frames. Depth and horizontal position are controlled in the world frame. The three forces of the loops, with the feed-forward force F_b that carries the net weight of the vehicle, are rotated into the body frame,

$$\mathbf{f}_{\text{body}} = \mathbf{R}^\top \begin{bmatrix} F_x \\ F_y \\ F_z + F_b \end{bmatrix}, \quad (14)$$

so that heave stays vertical when the vehicle is tilted. The attitude loops act on the Euler angles and use the body rates (p, q, r) as their rate feedback. The two agree only near level flight, which is where the vehicle is operated; a controller for large roll or pitch would have to use the attitude error in quaternion form.

Rate feedback. The rates are taken from the velocity that the simulator reports with each odometry sample. An earlier version differentiated the pose between messages; the arrival times jitter by 10 to 22 ms for samples 20 ms apart, which made the rate noisy and late, and the roll loop oscillated.

Hold and command. At every control step and for every degree of freedom, a non-zero command bypasses the outer loop: the command is the rate setpoint, and the pose setpoint is moved to the present pose. A zero command restores the outer loop, which then holds the pose at which the command ended. A command older than 0.5 s counts as zero, so that a lost operator link or a stopped guidance node results in a hold.

Modes. Four modes are provided, each adding to the previous one (Table 6). Their names are those of ArduSub. A change of mode clears all integrators and sets the setpoints to the present pose, with roll and pitch set to level. At start the depth setpoint is the depth at which the vehicle finds itself, or 1 m if it starts above that, since the vehicle of the open-water scene is created just above the surface.

Mode	Roll and pitch	Yaw	Depth	Horizontal position
Manual	open loop	open loop	open loop	open loop
Stabilise	held	held	open loop with F_b	open loop
Depth hold	held	held	held	open loop
Position hold	held	held	held	held, in the world frame

Table 6. Controller modes. In an open-loop axis the command is multiplied by a fixed force or torque per unit command, 120 N and 8 N m in the final configuration. The force was 100 N per unit during the first tests of Section 6.4.

6.4 Gain Design and Tuning

The gains were obtained in two stages. They were first calculated from the model of Eq. (7) with assumed parameters, so that the first closed-loop test would start from values of the right order. The parameters were then measured in the simulator, the assumptions that proved wrong were corrected, and the gains were adjusted by step tests on one axis at a time. Both stages are reported, because the differences between them show which properties of the vehicle the design is sensitive to.

6.4.1 Design Rules

Each axis has eight parameters (Eqs. (12) and (13)). They were chosen by the following rules, in this order.

Outer gain. If the inner loop tracks its rate setpoint, the pose error obeys $\dot{e} = -k_o e$ and decays with the time constant $T_o = 1/k_o$. The outer gain is therefore set directly by the time in which the axis should settle.

Rate limit. The rate limit $v_{\max}$ is the speed at which the vehicle approaches a distant setpoint. It is a matter of safety and of drag, not of stability.

Inner proportional gain. With proportional control only, Eq. (7) becomes $m\dot{v} + (d + k_p)v = k_p v_{\text{sp}}$, a first-order system with the time constant

$$T_i = \frac{m}{k_p + d} \approx \frac{m}{k_p} \quad (k_p \gg d). \quad (15)$$

A cascade behaves as designed only if the inner loop is clearly faster than the outer one. The rule adopted was a ratio T_o/T_i of about three, which gives $k_p \approx 3m k_o$.

Integral gain and clamp. The integrator removes what the proportional term cannot: a constant force such as the net weight or a current. Its corner frequency k_i/k_p was placed a factor of three to ten below the bandwidth $1/T_i$ of the inner loop, so that it does not reduce the phase margin there. The clamp $I_{\max}$ was then sized from the requirement that the largest force the integrator can supply, $k_i I_{\max}$, exceeds the steady disturbance expected on that axis.

Derivative gain. The derivative gain was set to zero. The water and the outer loop already damp the motion, and a derivative of the measured rate amplifies its noise.

Output limit. The output limit $\tau_{\max}$ of each axis is a share of the thrust that the thrusters common to several axes can deliver (Table 5), so that one axis cannot take the whole budget and leave the allocator to scale the others down.

Feed-forward. The force that is known in advance is supplied without waiting for an error: the net weight as a constant, and the drag $d v_{\text{sp}}$ in proportion to the rate setpoint.

6.4.2 First Estimates

The parameters assumed for the first calculation were the mass of the scene file, 14.7 kg, increased by 40 to 70 % for added mass to an effective 24 kg; the inertias of the rigid body, 0.15 and 0.22 kg m² in roll and pitch and about 0.4 kg m² in yaw; a net weight of about 5 N; and a thrust of 100 N per thruster. Table 7 gives the gains that follow from the rules.

The time constants express priorities. Roll and pitch were given the shortest, because an attitude error tilts the thrust of every other axis. Yaw was made slightly slower. Depth comes next, and horizontal position was made the slowest, being the least critical and, on the real

Axis	T_o	k_o	m or I	T_i	k_p	T_o/T_i	$v_{\max}$
x, y	1.25 s	0.8	24 kg	0.4 s	60	3.1	0.5 m/s
Depth	0.7 s	1.5	24 kg	0.3 s	80	2.3	0.5 m/s
Roll	0.25 s	4.0	0.15 kg m ²	0.1 s	2	2.5	2.0 rad/s
Pitch	0.25 s	4.0	0.22 kg m ²	0.1 s	4	2.5	2.0 rad/s
Yaw	0.33 s	3.0	0.4 kg m ²	0.13 s	3	2.5	1.5 rad/s

Table 7. First estimates of the loop gains, from assumed inertias. k_o in s⁻¹, k_p in N/(m/s) or N m/(rad/s). The gains were rounded. In pitch the rule gives 2.2; the value was set to 4 as an allowance for added inertia, which was not known at that stage.

vehicle, the least well measured. For depth the ratio of the two time constants is only 2.3, which was noted as tight at the time.

Table 8 gives the integral terms and output limits. The depth integrator was given an authority of 90 N, many times the assumed net weight, so that a current could also be absorbed. Depth was allowed 200 N, half of the nominal 400 N of the four vertical thrusters, with the other half reserved for roll and pitch.

Axis	k_i	$I_{\max}$	$k_i I_{\max}$	$\tau_{\max}$	Disturbance to be covered
x, y	10	5.0	50 N	120 N	drag of a current
Depth	30	3.0	90 N	200 N	net weight, assumed 5 N, and a current
Roll, pitch	2	2.0	4 N m	20 N m	offset between centres of mass and buoyancy
Yaw	1	2.0	2 N m	15 N m	asymmetry

Table 8. First estimates of the integral terms and output limits.

6.4.3 First Closed-Loop Tests

Two observations showed that these values were inadequate.

On a dive from the surface to a setpoint of 1 m the depth passed through 1.19 m, reached 1.42 m and returned to 1.00 m: an overshoot of 0.42 m on a step of that size. The outer loop demanded the full 0.5 m/s until the error fell below $v_{\max}/k_o = 0.33$ m, and the inner loop was too slow to remove that speed in the distance that remained.

In teleoperation, surge felt far faster than every other axis. In depth-hold mode surge is an open-loop force, which at that time was 100 N at full stick and gave 0.74 m/s. The axes under rate control did not reach their setpoints: yaw reached 14.7°/s of the 46°/s commanded, heave 0.22 m/s upwards and 0.28 m/s downwards of 0.5 m/s, and pitch 12.9°/s. An inner loop that settles at a third of its setpoint has a proportional gain that is small compared with the drag (Eq. (15)) and an integrator that is slow or exhausted, and both pointed to a plant different from the one assumed.

6.4.4 Identification of the Plant

The plant was therefore measured, in the reef scene, with a tool written for the purpose. It selects a mode, applies a constant command to one axis for a given time, and reports the rate reached, the times to 63 and 90 % of it, the peak thrust and the mean force or torque in the steady state. It can change controller parameters between steps, so that a series of gains is tested in one run. Three kinds of measurement were made.

- *Hover force.* In depth hold without a command, the steady heave force is the net weight

of the vehicle.

- *Drag*. At a constant rate the acceleration is zero and Eq. (7) reduces to $\tau = d v$, so the steady force divided by the rate is the drag. On the horizontal axes this was done in open loop with three forces; on the others in closed loop, by reading the force that the controller settled on.
- *Inertia*. In an open-loop step the rate approaches its final value with the time constant m/d , so $m = d T_{63}$.

Each series was run on a freshly started simulator, because open-loop runs leave the vehicle on the seabed, where contact falsifies later readings. Table 9 gives the results.

Quantity	Measurement	Result
Net weight	steady heave force in depth hold	75 N
Surge, sway	speed at 25, 50, 100 N: 0.21, 0.41, 0.74 m/s	$d \approx 120 \text{ N}/(\text{m/s})$, time constant 1.5 s
Heave	force at 0.28 and 0.56 m/s: 98 and 163 N	$d \approx 280 \text{ to } 350 \text{ N}/(\text{m/s})$
Yaw	torque at 0.4 and 0.8 rad/s: 6.2 and 12.7 N m	$d \approx 15 \text{ N m}/(\text{rad/s})$
Roll, pitch	torque at 0.4 rad/s	0.3 to 1 N m

Table 9. Plant measurements in simulation, from step tests. Heave forces are net of the force to hover.

The ratio of force to speed in surge is 119, 122 and 135 N/(m/s) at the three forces, so the drag is close to linear in the range of speeds flown, which supports the linear model of Eq. (7). Three results differ from the assumptions.

- *Net weight*. The vehicle weighs 145 N and sinks with 75 N, where about 5 N had been assumed. The depth integrator, with an authority of 90 N, was spending five sixths of it on holding the vehicle up, and what remained could not accelerate it upwards. This explains why heave was slower upwards than downwards.
- *Inertia*. A time constant of 1.5 s with a drag of 120 N/(m/s) gives an effective mass of about 180 kg, against 24 kg assumed, and the yaw response gives an inertia of about 17 kg m² against 0.4 kg m². Stonefish derives added mass from a simple shape fitted to the hull, which for an open frame encloses far more water than the frame displaces. With the measured mass and drag, Eq. (15) gives the first surge loop a time constant of $180/(60 + 120) = 1.0 \text{ s}$, not 0.4 s, which is no faster than the outer loop it was meant to serve.
- *Damping*. The drag is large compared with the first gains on the linear axes and in yaw. A proportional loop settles at the fraction $k_p/(k_p + d)$ of its setpoint until the integrator acts: one sixth in yaw and about one fifth in heave. Roll and pitch, in contrast, have almost no damping and little righting moment.

These values belong to the simulated vehicle and have not been compared with the hardware.

6.4.5 Retuning

The gains were then revised in five steps, each verified by a step test before the next.

Step 1: net weight. The feed-forward force F_b was set to 75 N upwards. The depth integrator then rests near zero in hover and its whole authority is available for disturbances. The benefit is largest after a change of mode, when the integrators are cleared: without the feed-forward the vehicle sinks until the integrator has built up 75 N again.

Step 2: drag feed-forward. The feed-forward gain of each axis was set to the measured drag: 280 in heave, 15 in yaw and 1 in pitch. On the horizontal axes the measured $120 \text{ N}/(\text{m/s})$ gave a rate overshoot, and the gain was reduced to 100, at which the overshoot was about 10 %. In roll every value above zero overshoot the rate, there being almost no drag to compensate, and the gain was left at zero.

Step 3: inner proportional gains. With the feed-forward carrying the steady load, the proportional gains were raised in steps, with the step test repeated at each value, until the commanded rate was reached quickly and the thrust showed no oscillation. The gains reached were 150 on the horizontal axes, 250 in heave, 6 in pitch and 30 in yaw, from one and a half times the first estimate in pitch to ten times in yaw, in keeping with the larger inertia. For surge, Eq. (15) now gives $180/(150 + 120) = 0.67 \text{ s}$, which agrees with the measured rise time of 1.6 s to 90 %, that is, 2.3 time constants. The ratio to the outer time constant of 1.25 s is then just under two. This is less than the factor of three aimed at, and it is the reason why the outer gains were not raised.

Step 4: integral terms. The integral gains were raised with the proportional ones so that the corner frequency k_i/k_p stayed between 0.3 and 1 s^{-1} . The clamps on the linear axes and in yaw were reduced to 1.0, since the feed-forward terms now supply what the integrators had to supply before, and a smaller clamp means less overshoot after a period of saturation. The remaining authority is 100 N in depth, 50 N on each horizontal axis, 4 and 12 N m in roll and pitch and 30 N m in yaw.

Step 5: output limits. With the thruster limit of 90 N the vertical thrusters deliver 360 N (Table 5). Depth was allowed 320 N , with the intention of leaving 40 N for roll and pitch. The feed-forward force F_b is added after this limit, however, so an upward demand at the limit exceeds the budget by 35 N and is then reduced by the scaling of the allocator together with the attitude torques; the limit should be lowered or applied after the feed-forward. The horizontal axes were raised to 150 N and yaw to 30 N m , in step with their gains.

Table 10 gives the first and the final values. The outer loops were left as designed.

Axis	Outer		Inner, first		Inner, final				
	k_o	$v_{\max}$	k_p	k_i	k_p	k_i	$I_{\max}$	$\tau_{\max}$	k_{ff}
x, y	0.8	0.5	60	10	150	50	1.0	150	100
Depth	1.5	0.5	80	30	250	100	1.0	320	280
Roll	4.0	2.0	2	2	2	2	2.0	20	0
Pitch	4.0	2.0	4	2	6	6	2.0	20	1
Yaw	3.0	1.5	3	1	30	30	1.0	30	15

Table 10. Controller gains. Units follow Eqs. (12) and (13): k_o in s^{-1} ; rates in m/s or rad/s ; forces in N and torques in N m . The derivative gain is zero on every axis.

6.4.6 The Roll Loop

Roll could not be treated like the other axes, and it limits the design. Its rigid inertia, 0.15 kg m^2 , is the smallest of the three, and it has almost no damping. The stability limit was found by experiment: the vehicle was held in hover, the inner proportional and integral gains were set to the same value, and the ripple of the roll rate was recorded (Table 11).

At a gain of 4 the loop is in a limit cycle, and the frequency of that oscillation is informative. The plant of the rate loop is close to a pure integrator, which contributes a phase lag of 90° ,

Inner roll gain $k_p = k_i$	2	3	4
Ripple of the roll rate in hover	0.0°/s	5.9°/s	478°/s, at 2.8 Hz

Table 11. Roll rate ripple in hover against the inner roll gain.

so the loop can only oscillate at the frequency at which its delays contribute the other 90°. At 2.8 Hz this is a delay of about 90 ms, between four and five periods of the 50 Hz loop. The chain from the sampling of the state through the controller, the allocator and the simulator interface to the next physics step accounts for part of it, and the load of rendering the reef for the rest. The roll gain is therefore bounded by latency and not by thrust. It was left at 2, the first estimate, with a margin of less than a factor of 1.5 to the onset of ripple. The same latency will exist on the vehicle in a different amount, so the roll gain has to be established again there.

6.4.7 Result

Table 12 gives the response to full-stick commands before and after tuning, and Table 13 the result of hands-off hold tests against the targets set for the controller. The full-stick commands themselves were revised with the gains, so that every axis responds at a comparable pace. The open-loop force per unit command was set to 120 N, the measured drag at 1 m/s, so that a command has approximately the meaning of a speed in every mode, and full stick in surge was made a command of 0.5. Full-stick surge in the open-loop modes is thereby 60 N instead of 100 N, which gives 0.49 m/s. The angular rates were set to values that the loops reach. Figure 13 summarises the effect of the tuning on rate tracking.

Axis	Command	Achieved before	Achieved after	Rise time (90 %)
Surge, position hold	0.5 m/s	not available	0.54 m/s	1.6 s
Heave	0.4 m/s	0.22 up, 0.28 down (of 0.5)	0.44 m/s	0.7 s
Yaw	40°/s	14.7°/s (of 46)	40.7°/s	0.3 s
Roll	23°/s	unstable at higher gain	23.0°/s	0.25 s
Pitch	23°/s	12.9°/s	24.2°/s	0.3 s

Table 12. Step response of the controller in simulation before and after tuning, single runs.

Mode	Depth	Roll	Pitch	Heading
Target	5 cm	2°	2°	3°
Depth hold	0.4 cm	0.03°	0.83°	0.31°
Position hold	3.2 cm	2.59°	1.12°	0.74°

Table 13. Largest deviation from the first sample during 30 s without commands, on ground-truth state, single runs.

Every axis now reaches its commanded rate within 10 %, and heave responds alike in both directions. Depth hold meets its targets with a wide margin. Position hold misses the roll target: sway and roll are coupled in the model (a sway command of 0.4 s produced 3° of roll in the test of Table 14), the position loop commands sway continually, and the weak roll loop is slow to remove the roll that results. An occasional roll disturbance of about 3° in hover, present with the first and the final gains alike, has not been explained.

The controller is adequate for teleoperation and for the slow survey flights of Section 10. It has not been tested against currents, thruster dynamics or sensor noise, its roll loop has little

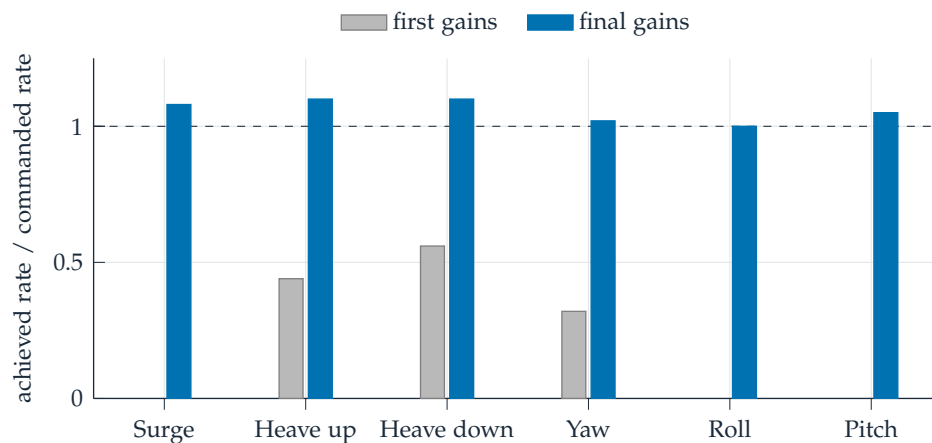


Figure 13. Rate reached at full stick as a fraction of the rate commanded, from Table 12. With the first gains the loops settled at a third to a half of their setpoints; with the final gains every axis is within 10% of it. No comparable figure exists for surge, roll and pitch with the first gains: surge was not under rate control, and the commanded rates in roll and pitch were changed between the two tests.

margin, and it should not yet be regarded as robust. The method carries over to the hardware as it stands: measure the net weight and the drag of each axis, set the feed-forward terms, raise the inner gains by step tests, and find the roll limit by experiment.

6.5 Verification of Signs

A sign error in one axis turns negative feedback into positive feedback, and the thruster directions of the model had already had to be corrected once (Section A). After the change of frames every axis was therefore checked by a test that starts from a level hover, switches to manual mode, applies a unit command on one axis for 0.4 s and records the displacement in the initial body frame (Table 14).

Command	Forward [m]	Starboard [m]	Down [m]	Roll [°]	Pitch [°]	Yaw [°]	Result
Surge	+0.022	0.000	+0.045	0.0	+0.8	0.0	forward
Sway	0.000	+0.021	+0.053	-3.1	0.0	0.0	to starboard
Heave	-0.002	0.000	+0.074	-0.1	-0.2	+0.1	down
Roll	0.000	-0.003	+0.054	+36.3	0.0	0.0	starboard down
Pitch	+0.001	0.000	+0.054	-0.1	+12.8	0.0	nose up
Yaw	0.000	0.000	+0.055	-0.1	0.0	+2.9	clockwise

Table 14. Response to a positive unit command on each axis for 0.4 s in manual mode. The descent of about 0.05 m in every row is the vehicle sinking under its net weight, which manual mode does not compensate.

All six axes respond with the intended sign. The table also shows the two couplings that the model of independent axes ignores, sway into roll and surge into pitch, and the difference in inertia between the angular axes: the same torque command turned the vehicle 36° in roll, 13° in pitch and 3° in yaw.

6.6 Alternatives Considered

The cascaded PID controller was chosen as a baseline, not as the final design. It needs no model, it can be tuned on the vehicle, and it shares its structure with ArduSub. Its weaknesses

are those seen above: the axes are tuned separately although they interact, the gains suit one operating point, and a current is rejected only as fast as the integrator acts. Table 15 lists the methods that were examined as successors and what each requires.

Method	Benefit	Requirement or drawback
Model feed-forward added to the PID	feed-forward carries most of the load	added mass and drag of Eq. (6), identified in a pool
Disturbance observer [12]	rejects a steady current or tether pull faster than an integrator	an approximate inertia per axis; can be added to any of the others
Sliding mode control	robust to bounded model error	chattering of the thrusters unless a boundary layer or a second-order form is used
Linear quadratic regulator	tunes all axes together, with their coupling	a model linearised at hover; valid near hover only
Model predictive control	handles thruster limits and allocation in one problem	a good model and an optimiser on the on-board computer

Table 15. Control methods examined as successors of the baseline.

The measurements of Table 9 are the first step towards the first of these, and a disturbance observer is the most likely next addition, because it needs the least model and addresses the weakness that matters most for holding position in a current.

6.7 Teleoperation

A gamepad or a keyboard drives the vehicle through the controller. Stick deflections are turned into velocity commands in the body frame, with full deflection giving 0.5 m/s in surge, 0.4 m/s in sway and heave, and 0.7 rad/s in yaw, and buttons select the mode and level the vehicle. Because the commands are velocities and a zero command means “hold”, the same interface serves the guidance of Section 10: an autonomous flight is a sequence of velocity commands that no pilot has to give.

7 State Estimation

7.1 Purpose and Status

The controller needs the pose and the velocity of the vehicle [8]. In simulation both are available exactly, and all results of the other sections use this ground truth. The vehicle itself will carry an inertial sensor, a pressure sensor and cameras, and no Doppler velocity log, so an estimator is needed that produces the same state from those sensors. I began this work on 23 September 2026. It is kept on a separate branch of the repository, it has been run in the simulator, and it is not complete: the controller holds depth and attitude on the estimate, but not heading or horizontal position to the standard reached on ground truth. This section describes the design, the measurements made so far and what they show to be missing.

7.2 Filter

The estimator is the extended Kalman filter of the `robot_localization` package [14], running at 50 Hz. Its state has fifteen components,

$$\mathbf{x} = (x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, p, q, r, \ddot{x}, \ddot{y}, \ddot{z}), \quad (16)$$

and its process model is a generic rigid body moving with constant acceleration and constant angular rate over one step. It contains nothing of Eq. (6): neither drag nor the commanded wrench enters the prediction. The filter was chosen because it is an established implementation that accepts any combination of pose, velocity and acceleration measurements, each with its own covariance, and because its omission of the vehicle dynamics makes it independent of a model that has not been identified. The cost of that omission appears below. Table 16 lists the measurements that are fused.

Input	Rate	States measured	Noise in the scene (standard deviation)
Inertial sensor	50 Hz	ϕ, θ, ψ	0.01, 0.01, 0.03 rad; heading drift 0.001 rad/s
		p, q, r	0.005 rad/s
		$\ddot{x}, \ddot{y}, \ddot{z}$	0.05 m/s ²
Pressure sensor	50 Hz	z	20 Pa, about 2 mm
Position stand-in	5 Hz	x, y	0.10 m

Table 16. Measurements fused by the filter. No input measures a linear velocity.

The noise of the inertial sensor in Table 16 was set for this work. The scenes on the main branch carry larger values, 0.1 rad in roll and pitch and 0.5 rad in heading, which are far worse than any attitude reference and had gone unnoticed because nothing had used that sensor until then (Section 8.7 gives their measured effect).

7.3 Measurement Models

A small adapter node converts the output of the simulated sensors into the measurements of Table 16, in the frames of Section 3.2. On the vehicle it is replaced by the driver of the inertial sensor with its mounting rotation, a pressure converter and the position source.

Depth. The simulated sensor reports gauge pressure, which was confirmed by a reading of 10 114 Pa at a depth of 1.00 m. Depth and its standard deviation follow from the hydrostatic relation,

$$z = \frac{P}{\rho g}, \quad \sigma_z = \max\left(\frac{\sigma_P}{\rho g}, 0.01 \text{ m}\right), \quad (17)$$

with $\rho = 1031 \text{ kg/m}^3$, the density of the water in the scene. The floor on the standard deviation keeps the filter from trusting depth to the millimetre, which the hydrostatic relation does not justify in moving water.

Acceleration. An accelerometer measures specific force, not acceleration: at rest the simulated sensor reads $+9.74 \text{ m/s}^2$ along the upward axis of the model. With $\tilde{\mathbf{a}}$ the specific force in the body frame and $\mathbf{R}$ the attitude reported by the same sensor, the kinematic acceleration is

$$\mathbf{a} = \tilde{\mathbf{a}} + \mathbf{R}^\top \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix}. \quad (18)$$

The filter can remove gravity itself, but its implementation assumes a world frame with z up. The removal is therefore done in the adapter, in the NED frame, and switched off in the filter.

Equation (18) makes the horizontal acceleration sensitive to the attitude. An attitude error δ leaves a component $g \delta$ of gravity in the horizontal acceleration. For $\delta = 0.01 \text{ rad}$ this is about 0.1 m/s^2 , which, integrated twice, is a position error of 1.2 m after 5 s. The inertial sensor alone therefore cannot carry the horizontal position for more than a second or two between position fixes, and this, not the noise of the accelerometer, decides how often fixes are needed.

Frames and covariances. Angular rate and specific force are rotated into the FRD frame with the matrix $\mathbf{S}$ of Eq. (3), and their covariances with $\mathbf{S}^\top \boldsymbol{\Sigma} \mathbf{S}$. The covariance of the orientation is passed on unchanged. Stonefish adds its orientation noise to the Euler angles of the model in the world frame, and for a model that is stood upright these are the roll, pitch and heading of the FRD frame, so the diagonal already has the required meaning.

7.4 Observability

Three groups of states have to be distinguished.

Depth, roll and pitch are measured directly and without drift: depth by pressure, roll and pitch by the direction of gravity. These are also the states that depth hold and attitude hold need.

Heading is measured by the inertial sensor, but in the scene, as on a vehicle whose magnetometer is disturbed by its thrusters, it drifts. The scene specifies a drift of 0.001 rad/s, about 3.4° per minute, and nothing in Table 16 can correct it. The filter follows the drifting heading and the controller holds it faithfully, so the true heading turns at the drift rate.

Horizontal position and velocity are not observable from the inertial and pressure sensors at all. For the time being the filter receives the true horizontal position with Gaussian noise of 0.10 m at 5 Hz, as a stand-in for a position sensor, so that the remainder of the chain could be built and tested. Linear velocity is measured by nothing; the filter infers it from the sequence of position fixes and from the acceleration. This is the weakest point of the design, because the inner loops of the controller (Eq. (13)) act on exactly that velocity.

7.5 Integration with the Controller

The controller has a parameter that selects its source of state: the ground truth, converted by Eq. (3), or the output of the filter, which is already in the FRD and NED frames. Nothing else in the controller changes. With the filter as source the controller waits until it has received estimates for one second before it sets its hold setpoints, since the first estimates precede the first position and depth measurements and would otherwise be latched as the pose to hold.

7.6 Measurements

Table 17 gives the result of hold tests of 60 s with the loops closed on the estimate, in the reef scene, and for comparison the tests of Table 13 on ground truth. The deviations are those of the true pose. Table 18 gives the error of the estimate itself during the same runs.

Mode	State used	Depth	Roll	Pitch	Heading	Horizontal
Target		5 cm	2°	2°	3°	
Depth hold	ground truth	0.4 cm	0.03°	0.83°	0.31°	
Depth hold	estimate	0.8 cm	0.40°	0.80°	3.29°	1.2 cm
Position hold	ground truth	3.2 cm	2.59°	1.12°	0.74°	
Position hold	estimate	0.8 cm	3.51°	1.53°	5.21°	18.8 cm

Table 17. Largest deviation of the true pose during a hold without commands, after the first 5 s. Runs on the estimate lasted 60 s, those on ground truth 30 s; single runs. Horizontal deviation was not recorded in the earlier runs.

The two sets of runs are not strictly comparable: those on ground truth lasted 30 s and were measured from the first sample, those on the estimate lasted 60 s and exclude the first 5 s. The comparison is indicative, and Fig. 14 shows it relative to the targets.

Mode	x	y	Depth	Roll	Pitch	Heading
Depth hold	7.4 cm	6.9 cm	0.2 cm	0.52°	0.50°	4.77°
Position hold	8.4 cm	7.1 cm	0.2 cm	0.51°	0.51°	9.99°

Table 18. Root-mean-square difference between the estimate and the ground truth in the runs of Table 17.

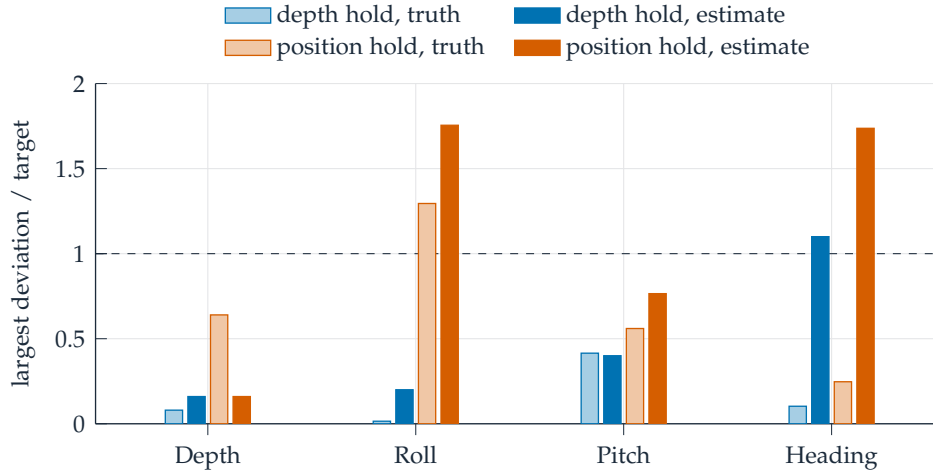


Figure 14. Hold deviations of Table 17 as fractions of their targets (dashed line). On the estimate, depth, and roll and pitch in depth hold, stay within target; heading exceeds it in both modes because of the drift of the inertial heading, and roll in position hold exceeds it on either source of state.

The tables confirm the three groups of Section 7.4.

Depth is estimated to 2 mm and held to 8 mm; roll and pitch are estimated to 0.5°, which is the noise of the sensor, and held within their targets in depth hold. For these states the estimate can replace the ground truth today.

The heading error of the estimate grew from 4.4° to 7.2° during the 60 s of the depth-hold run, a rate of 0.0008 rad/s, which is the drift written in the scene. The heading target is missed for this reason alone. It cannot be met without an absolute heading reference, and the marker fixes of Section 8, which measure heading to better than 1°, are the obvious candidate.

In position hold the vehicle wandered by 19 cm and rolled by 3.5°. The horizontal estimate is accurate to about 8 cm, slightly better than the fixes it is built from, but it moves with their noise, and so does the velocity that the filter derives from them. The position loop turns this into sway commands, and sway couples into roll (Table 14), the axis with the least margin. The result is not a failure of the filter but a mismatch between a controller that was tuned on a noise-free velocity and an estimate that has none to offer.

7.7 Fusion of the Marker Fixes

The marker localization of Section 8 publishes a pose with a covariance in the frames of the filter, so replacing the stand-in requires only that the position input of the filter is pointed at it, with heading enabled in addition to x and y . This has been prepared and not yet run. Three properties of the fixes make it more than a change of configuration.

- Fixes arrive only when a board is in view, within 10 m. Between boards the filter has no position input, and by the argument following Eq. (18) its horizontal estimate will be useless within seconds. Either a velocity measurement or a model of the vehicle dynamics in the prediction is needed to bridge the gaps.

- The uncertainty of a fix varies by more than an order of magnitude with range (Table 19). The filter weights each fix by its covariance, which is why the covariance was verified, but a gate on the Mahalanobis distance of the innovation has to be configured so that a wrong fix is rejected and not averaged in.
- A fix describes the moment at which its image was taken and arrives later. The filter has to be configured to insert it at its time stamp.

7.8 What Remains

In order of importance: a source of horizontal velocity, or a prediction that uses the commanded wrench and the drag of Table 9; the fusion of the marker fixes with an innovation gate and with their heading; a retuning of the horizontal loops, with lower gains or a filter on the velocity, for a state that carries noise; and a timeout in the controller, which at present continues on the last state it received if the estimate stops arriving. Until these are done the controller is flown on ground truth, and the results of the following sections must be read with that in mind.

8 Vision-Based Localization

8.1 Problem

The inertial sensor gives attitude and the pressure sensor gives depth, but nothing on the vehicle measures where it is horizontally. Position hold therefore needs an external reference. The simplest reference that a single camera can use is a set of fiducial markers at surveyed positions. Markers also give a metric scale, which a single camera cannot recover from natural features alone.

8.2 Marker Map

Fourteen boards carrying AprilTag markers of the 36h11 family [15], a kind of marker that has been evaluated under water [17], are placed in two rings around the reef mound, facing it, 1.2 m above the seabed. Each black square measures 0.6 m. One file lists the identifier, position and facing direction of every board. The localization node reads that file, and the boards in the simulator scene are generated from the same file, so the two cannot disagree.

8.3 Camera Model

The camera is modelled as a pinhole without distortion, which is exact for the simulated camera. With an image of 1280×720 pixels and a horizontal field of view of 80° , the focal length is $f = 640 / \tan 40^\circ = 763$ pixels and the principal point is the image centre. A world point $\mathbf{X}$ is seen at the pixel

$$\pi(\mathbf{X}; \boldsymbol{\xi}) = \begin{bmatrix} f X_c / Z_c + c_x \\ f Y_c / Z_c + c_y \end{bmatrix}, \quad \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \mathbf{R}_{bc}^\top \left(\mathbf{R}(\boldsymbol{\xi})^\top (\mathbf{X} - \mathbf{p}) - \mathbf{t}_{bc} \right), \quad (19)$$

where $\boldsymbol{\xi} = (x, y, z, \phi, \theta, \psi)$ is the pose of the vehicle, $\mathbf{p}$ its position, $\mathbf{R}(\boldsymbol{\xi})$ its attitude, and $\mathbf{R}_{bc}$, $\mathbf{t}_{bc}$ the mounting of the camera on the body. The mounting is read from the file that also generates the scenes (Section 5.4). A marker with a side of $s = 0.6$ m at range r spans fs/r pixels: 153 at 3 m and 51 at 9 m.

8.4 Pose Estimation

The markers are detected with the ArUco module [16] of OpenCV [30], with sub-pixel refinement of the corners. Let $\mathbf{X}_j$ be the surveyed world position of corner j and $\mathbf{x}_j$ its detected pixel. The vehicle pose is found by minimising

$$E(\xi) = \sum_j \frac{\|\pi(\mathbf{X}_j; \xi) - \mathbf{x}_j\|^2}{\sigma_{\text{px}}^2} + \frac{(\phi - \phi_{\text{imu}})^2 + (\theta - \theta_{\text{imu}})^2}{\sigma_{\text{att}}^2}, \quad (20)$$

with $\sigma_{\text{px}} = 0.5$ pixels and $\sigma_{\text{att}} = 2^\circ$. The first term is the reprojection error of the corners of all visible markers, so that every marker in view contributes to one pose. The second term is a prior on roll and pitch from the inertial sensor. The problem is solved by the Levenberg-Marquardt method [20, 21].

Starting points and the planar ambiguity. Four coplanar points seen from a distance admit two poses that fit almost equally well [18, 19]: the true one, and one in which the marker is tilted the opposite way about the line of sight. A minimisation started near one of them does not find the other. For every visible marker the pose of the marker relative to the camera is therefore computed in closed form, its mirrored counterpart is constructed, and Eq. (20) is minimised from both. Solutions that coincide are merged, and the remaining ones are ranked by their reprojection error. The closed-form solver for planar targets that OpenCV provides was tried for this step first and returned grossly wrong poses in testing, so the general iterative solver is used with the mirrored solution constructed explicitly.

8.5 Why a Single Marker Is Weak at Range

The need for the prior, and the limit of what it achieves, follow from a short calculation. Consider one marker seen head-on at range r and rotated by a small angle α about its vertical axis. Its near and far vertical edges lie at ranges $r \mp \frac{1}{2}s \sin \alpha$ and appear with heights $fs / (r \mp \frac{1}{2}s \sin \alpha)$, which differ by

$$\Delta h \approx \frac{fs^2}{r^2} \sin \alpha. \quad (21)$$

The width of the marker changes with $\cos \alpha$ and carries no first-order information at $\alpha = 0$, so Δh is the only observation of the rotation. It is formed from four corner coordinates, each with a standard deviation σ_{px} , and therefore has a standard deviation of $2\sigma_{\text{px}}$. Hence

$$\sigma_\alpha \approx \frac{2\sigma_{\text{px}} r^2}{fs^2}, \quad \sigma_\perp \approx r \sigma_\alpha = \frac{2\sigma_{\text{px}} r^3}{fs^2}, \quad (22)$$

where $\sigma_\perp$ is the uncertainty of the camera position across the line of sight: the direction to the marker is known to a fraction of a pixel, so a camera that misjudges the orientation of the marker by α places itself on the wrong point of a circle of radius r around it. The position error grows with the cube of the range. The same holds for a rotation about the horizontal axis and the vertical position.

Table 19 gives the standard deviations that the estimator itself reports for this configuration, computed from its covariance (Section 8.6), and Fig. 15 compares them with Eq. (22). The estimator reports values 4 to 12 % above the formula and follows the same cubic growth. Range is known well at every distance, because it is observed through the size of the marker, a first-order effect.

The table shows what the prior does and what it does not do. Roll and pitch fix the rotation of the camera about the two horizontal axes, and with it the vertical position, whose uncertainty at 9 m falls from 2.75 to 0.32 m. They say nothing about the rotation about the vertical axis,

Range	Marker side	Without prior		With prior	
		across, horizontal	vertical	across, horizontal	vertical
3 m	153 px	0.11 m	0.11 m	0.11 m	0.08 m
4 m	115 px	0.25 m	0.25 m	0.25 m	0.13 m
6 m	76 px	0.83 m	0.83 m	0.83 m	0.21 m
9 m	51 px	2.75 m	2.75 m	2.75 m	0.32 m

Table 19. Standard deviation of the position from a single marker seen head-on by a level camera, with a corner noise of 0.5 pixels, without and with a prior of 2° on roll and pitch. Along the line of sight the standard deviation is 0.01 to 0.06 m in all cases. Computed from the covariance of the estimator, not measured.

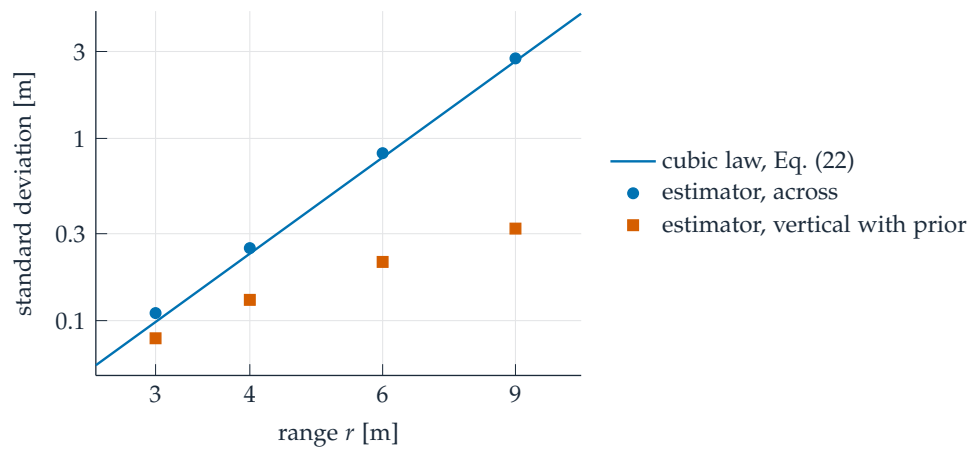


Figure 15. Uncertainty of a single-marker fix against range. Across the line of sight it follows the cubic law of Eq. (22) whether or not the attitude prior is used. The prior bounds only the vertical component, which then grows roughly linearly with range.

so the horizontal uncertainty across the line of sight is unchanged. That component could be constrained only by a heading reference of comparable quality, which the vehicle does not have (Section 7.4), or by a second marker in view at a different bearing. The heading of the inertial sensor is used for one purpose only: to choose between two solutions that fit equally well, provided that one of them lies within 30° of it.

Two consequences were drawn. Distant markers are not used at all (Section 8.6), and the covariance is published with every fix, so that a filter gives a fix at 9 m several hundred times less weight than one at 3 m. In practice the detected corners in the simulator are better than half a pixel, and the errors measured at 4 m (Table 20) are about four times smaller than the table predicts.

An earlier version fixed roll and pitch to the inertial values exactly instead of weighting them. That failed when the markers were close: an attitude error of one degree rotated the predicted corners by several pixels, the fit exceeded its error threshold and good fixes were discarded. Treating the inertial attitude as a measurement with its own uncertainty removed the problem, since close markers then outweigh it.

8.6 Covariance and Gating

The covariance of the fix is that of the weighted least-squares problem,

$$\Sigma_{\xi} = \left(\mathbf{J}_r^\top \mathbf{J}_r \right)^{-1}, \quad (23)$$

where $\mathbf{J}_r$ is the Jacobian of the residuals of Eq. (20), each divided by its standard deviation. The covariance therefore grows with range and viewing angle and shrinks when several markers are visible, and a filter that fuses the fixes can weight them correctly. A unit test confirms that the predicted standard deviations agree with the scatter of repeated noisy fits in all six components to within 30 %. The published standard deviations are not allowed to fall below 0.02 m and 0.5° , because the survey of the boards and the mounting of the camera are not known better than that and the fit knows nothing of them.

A fix is discarded in four cases: when the reprojection error exceeds 2 pixels; when two different poses fit almost equally well, which happens for a single marker seen head-on, and the inertial heading cannot decide between them; when the marker is more than 10 m away; and when a marker is smaller than 45 pixels in the image, which corresponds to a range of about 10 m. The last rule was added after a distant marker, about 40 pixels wide in the image, produced a fix that was 5 m wrong with a small reported uncertainty.

8.7 Validation

The method was validated in three stages, each of which found faults that the previous one could not.

Unit tests. Synthetic corner observations with known vehicle poses confirm that the pose is recovered exactly without noise, that the covariance matches the scatter with noise, and that ambiguous and badly fitting observations are rejected. One test renders the marker boards into a synthetic image and runs detection and pose estimation end to end.

Node test without the simulator. The localization node was started from its launch file and supplied with rendered camera images, inertial data and odometry over the ROS 2 topics it uses in operation. This exposed the two faults described above, the rigid attitude constraint and the acceptance of distant markers, before any simulator run.

Simulator. In Stonefish the vehicle flew to a position 4 to 5 m in front of one board and moved 1 m from side to side (Fig. 16), while the fixes were compared with the ground truth for 40 s. Table 20 gives the four runs that were made. The target was a position error below 0.10 m and a heading error below 3° .

Run	Fixes	Position error		Heading error, mean	Within 2σ
		mean	95 %		
Level camera, noise-free inertial sensor	300	0.055 m	0.070 m	0.42°	100 %
Level camera, inertial noise of the default scenes	454	0.121 m	0.255 m	0.51°	95 %
Camera pitched 20° down, vehicle parallel to the board	505	0.102 m	0.123 m	0.95°	100 %
Camera pitched 20° down, vehicle facing the board	269	0.052 m	0.059 m	0.57°	99 %

Table 20. Marker localization against ground truth in Stonefish, one board at about 4 m, single runs of 40 s. The last column is the share of fixes whose error lies inside twice the reported standard deviation; about 95 % is expected. The last run was made in the 150-coral reef and contains one outlier of 0.47 m.



Figure 16. Forward camera during the last run of Table 20. The near board is detected and used; the board in the background is too small in the image and is ignored.

Four things follow from the table. The method meets its target when the vehicle faces the board, at a range at which Table 19 gives it a comfortable margin; nothing has been measured beyond 5 m, where the calculation of Section 8.5 predicts a rapid loss. With the camera pitched down for the mapping work (Section 9.11) and the vehicle parallel to the board, the board is seen obliquely near the top of the image and the error doubles. The reported uncertainty is honest in all runs. And the method tolerates a poor attitude prior: the inertial sensor of the default scenes has, measured against the ground truth, an attitude error with a standard deviation of 5.5° in roll and pitch and 28° in heading, because the noise values in the scene file are interpreted in radians, yet the fixes at 4 m remain usable, since close markers outweigh the prior as intended.

In the runs with a moving vehicle the error is nearly constant rather than scattered around zero. The comparison pairs each fix with the latest ground-truth sample at the moment the fix arrives, and the vehicle moved at up to 0.3 m/s, so a processing delay of about 0.15 s would account for it. This has not been confirmed by a stationary test.

The first simulator run also showed that the boards were rendered black. Their mesh had been written without surface normals, which Stonefish needs for shading, and their textures as single-channel images, which it does not accept. Both were corrected in the scene generator.

9 Coral Perception and Mapping

9.1 Task

The perception pipeline finds corals in the camera image, follows each one from frame to frame, estimates where it stands in the world and maintains a map of the corals seen so far. The vehicle carries a single camera in each direction and no sonar, so range has to be inferred from the motion of the vehicle. The pipeline has four stages (Fig. 17) and only produces estimates; it never commands the vehicle.

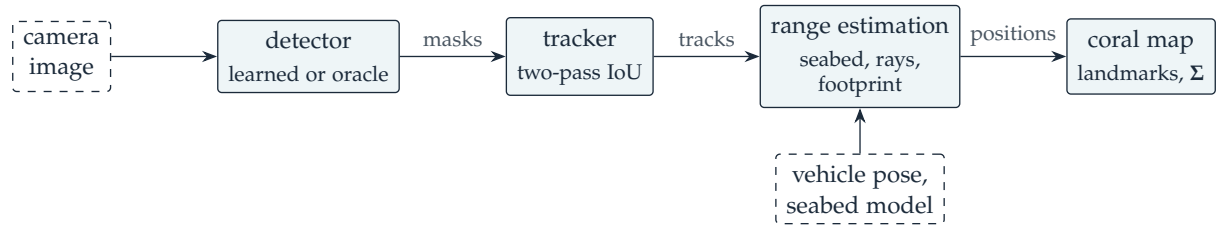


Figure 17. Stages of the perception pipeline. Dashed boxes are inputs. The detector is the only stage that depends on appearance; the three later stages are geometric and were developed against the oracle detector.

9.2 Detection

The detector returns, for each coral in the image, its family, a confidence and a pixel mask. The nine families of the reef scene are the classes. Two interchangeable detectors were built.

The *learned* detector is an instance segmentation network of the YOLO family [22, 23], fine-tuned on images from the simulator. It is the detector intended for the real vehicle.

The *oracle* detector reads the segmentation camera of the simulator and therefore returns perfect masks. It needs no trained model and no graphics card for inference, and it allows the three later stages to be developed and tested on their own.

9.3 Automatic Labelling

Training a segmentation network needs a mask for every coral in every training image. These were not drawn by hand. The segmentation camera gives a mask for each object, but it identifies the objects by numbers that the simulator assigns in an order which the scene file does not state. The correspondence between numbers and corals is therefore discovered from the images themselves.

For each coral a set of points is sampled from the surface of its mesh. In every frame the points within 8 m of the camera are projected into the segmentation image with the true pose of the vehicle (Eq. (19)), and each point casts a vote for the object number found at its pixel. Let n_{ik} be the number of votes that the points of coral i have cast for object number k over all frames so far. Number k is assigned to the coral $i^* = \arg \max_i n_{ik}$ when it has received at least 20 votes from that coral and one of two conditions holds:

$$\frac{n_{i^*k}}{\sum_i n_{ik}} \geq 0.7 \quad \text{or} \quad \frac{n_{i^*k}}{\sum_i n_{ik}} \geq 0.4 \quad \text{and} \quad k = \arg \max_{k'} n_{i^*k'}. \quad (24)$$

The first condition says that nearly all votes for the number come from one coral. The seabed, whose number is hit by the hidden points of every coral, never comes near it. The second condition accepts a smaller share when coral and number are each other's first choice. Both the use of surface points and the second condition were introduced because a simpler rule failed (Section 9.9).

Each training image is additionally altered at random in the ways in which water alters an image: the red channel is attenuated by up to 60 %, and green and blue by less; the image is blended by up to 50 % with a blue-green veil that imitates backscatter; it is blurred, and sensor noise is added. The geometry is not changed, so the masks remain valid. The intention, as in domain randomisation [29], is to reduce the dependence of the network on the particular appearance of the simulator.

9.4 Tracking

Detections are associated across frames by the overlap of their bounding boxes, in two passes in the manner of ByteTrack [26]. The overlap of two boxes A and B is measured by their intersection over union (IoU),

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad (25)$$

and in each pass the assignment of detections to tracks that maximises the total overlap is found with the Hungarian algorithm [24]. Before the comparison the box of a track is moved by the velocity of its centre in the image, which is estimated from its past matches, so that a coral that was missed for a few frames is looked for where it has moved to.

In the first pass, detections with a confidence of at least 0.5 are matched to the existing tracks, and a pair is accepted when its overlap is at least 0.2. In the second pass the tracks that remained unmatched are offered the detections with a confidence between 0.1 and 0.5, and a pair needs an overlap of 0.4. This keeps the identity of a coral through frames in which the detector is unsure, without allowing weak detections to start tracks: a new track is started only by an unmatched detection with a confidence of at least 0.6. A track is confirmed after three matches and is dropped after thirty frames without one.

The class of a track is the confidence-weighted majority of its detections. A detection of a different class is matched to a track only when the overlap is at least 0.6. A detector that alternates between two families on the same coral then still matches, while a track is prevented from jumping to an adjacent coral of another family.

9.5 Range from a Single Camera

A single image gives the direction to a coral and not its distance. For a pixel $\mathbf{x}$ and a vehicle pose ξ , inverting Eq. (19) gives a ray in the world with origin $\mathbf{o}$ at the camera and unit direction $\mathbf{d}$. Three sources of range are combined.

Seabed intersection. A coral stands on the seabed, so the ray through the bottom centre of its bounding box meets a model of the seabed at the coral. The intersection is found by stepping along the ray in increments of 0.25 m until it passes below the seabed and refining by bisection. This gives a position from the first frame. It is as accurate as the seabed model and the depth of the vehicle allow, and it is assigned a standard deviation of 20 % of the range.

Triangulation. As the vehicle moves, rays to the same coral from different positions intersect [27]. Ray i constrains the coral position $\mathbf{c}$ in the two directions across it. With $\mathbf{P}_i = \mathbf{I} - \mathbf{d}_i \mathbf{d}_i^\top$ the projection onto the plane normal to the ray, the estimate minimises the weighted squared

distances from the rays,

$$\hat{\mathbf{c}} = \arg \min_{\mathbf{c}} \sum_i w_i \|\mathbf{P}_i(\mathbf{c} - \mathbf{o}_i)\|^2, \quad w_i = \frac{1}{(r_i \sigma_{\alpha,i})^2}, \quad (26)$$

where r_i is the range along ray i and $\sigma_{\alpha,i}$ its angular standard deviation, so that $r_i \sigma_{\alpha,i}$ is the uncertainty of the ray in metres at the coral. Setting the gradient to zero gives the linear system

$$\left(\sum_i w_i \mathbf{P}_i \right) \hat{\mathbf{c}} = \sum_i w_i \mathbf{P}_i \mathbf{o}_i, \quad \Sigma_{\mathbf{c}} = \left(\sum_i w_i \mathbf{P}_i \right)^{-1}. \quad (27)$$

Since the weights depend on the ranges and the ranges on the solution, the system is solved three times with updated weights.

The quality of the result depends on the angle that the rays span. For two rays at range r that meet at an angle β , Eq. (27) gives the standard deviations across and along the mean viewing direction,

$$\sigma_{\text{across}} = \frac{r \sigma_{\alpha}}{\sqrt{2} \cos(\beta/2)}, \quad \sigma_{\text{along}} = \frac{r \sigma_{\alpha}}{\sqrt{2} \sin(\beta/2)}. \quad (28)$$

With $\sigma_{\alpha} = 1^\circ$ at 3 m the uncertainty across is 0.04 m whatever the angle, while along the viewing direction it is 0.85 m at $\beta = 5^\circ$ and 0.14 m at $\beta = 30^\circ$ (Fig. 18). A triangulated position is therefore accepted only from at least three rays that span 5° or more, and its covariance decides when it is good enough to be published (Section 9.6). A new ray is stored only when the camera has moved 0.15 m since the last one, at most forty are kept, and when the limit is reached a ray from the middle of the sequence is discarded, because the first and the latest views span the baseline.

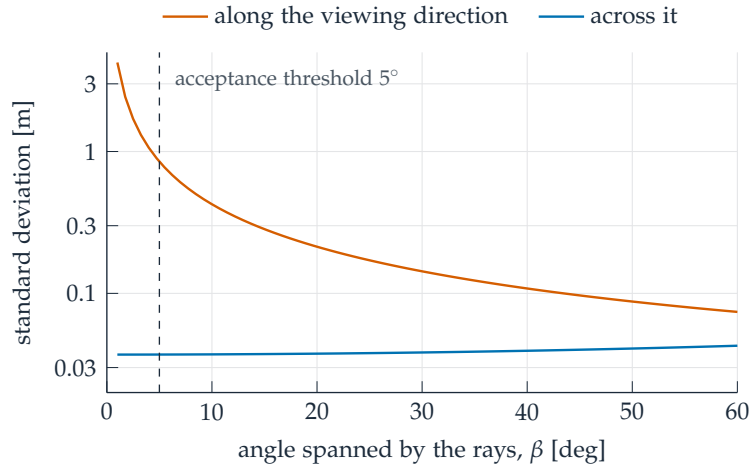


Figure 18. Uncertainty of a position triangulated from two rays at 3 m with $\sigma_{\alpha} = 1^\circ$, from Eq. (28). The component along the viewing direction is governed by the parallax; the component across it is not.

The ray is taken through the centre of the mask. Seen from the side, the centre of a large mask is a different point of the coral from every direction, so the rays do not meet in one point. The angular uncertainty of such a ray is therefore raised from 1° to 20% of the angular size of the mask. From above, the centre of the mask stays over the middle of the coral and no such allowance is made.

Footprint. A coral that is cut by the edge of the image has neither a visible base nor a meaningful centre. From the survey height this is the usual case for the downward camera, and a table coral five metres wide is never seen whole. When the camera looks down, 24 points

of the outline of the mask are therefore projected onto the seabed model in every frame, and the cells of a grid of 0.2 m that they fall into are accumulated over the life of the track. After five frames the coral is placed at the middle of the extent of the covered cells, not at their mean, so that a part of the coral that was in view for longer does not attract the estimate. The position is assigned a standard deviation of 0.35 m.

The projection assumes that the outline lies on the seabed. A point of the outline that is in fact at a height h above the seabed, seen at an angle γ from the vertical, is placed

$$\Delta = h \tan \gamma \quad (29)$$

too far from the point below the camera: 0.29 m for a coral 0.5 m high seen 30° off the vertical. When the coral is passed on both sides, on adjacent legs of a survey, the footprint is enlarged on both sides and its middle is hardly moved; when it is seen from one side only, half of Δ remains as a bias. This is of the order of the position errors measured below and is the principal limit of the method.

Order of preference. Triangulation is used when it is available, the footprint otherwise, and the single seabed intersection only until one of the two exists. To every estimate the uncertainty of the vehicle position, taken as 0.10 m, is added.

9.6 Map

A track ends when a coral leaves the image, and a new track starts when it is seen again, on the next leg of the survey or after a detector failure. The map therefore attaches each track to a landmark. An estimate with position $\mathbf{c}$ and covariance Σ agrees with a landmark at $\mathbf{c}_L$ with covariance Σ_L when one of three conditions holds: the two are of the same class and their footprints share at least three cells; or they are less than 0.8 m apart; or they are less than 3.2 m apart and consistent within their uncertainties,

$$(\mathbf{c} - \mathbf{c}_L)^\top (\Sigma + \Sigma_L)^{-1} (\mathbf{c} - \mathbf{c}_L) \leq 3^2. \quad (30)$$

The third condition lets a rough first estimate join the coral it belongs to, and the limit on the distance keeps a very uncertain estimate from joining a coral far away. A track that agrees with no landmark starts a new one.

The position of a landmark is computed from the latest estimates of its tracks. If any of them is triangulated, only the triangulated ones are used and are combined by their information,

$$\Sigma_L = \left(\sum_t \Sigma_t^{-1} \right)^{-1}, \quad \mathbf{c}_L = \Sigma_L \sum_t \Sigma_t^{-1} \mathbf{c}_t. \quad (31)$$

Otherwise, if there are footprints, the landmark is placed at the middle of their union, which is how a coral wider than the strip seen by the camera is mapped as one object from several legs. Two corrections are applied as estimates improve. A track that was attached to a neighbouring coral on the strength of a rough first estimate is detached once its triangulated position lies more than 0.8 m from those of the other tracks of that landmark. And two landmarks whose triangulated positions have converged to within 0.8 m, or whose footprints of the same class have come to overlap, are merged.

A landmark is published when it has been observed at least five times and the largest standard deviation of its position is below 0.5 m. Its class is the confidence-weighted majority of the classes of its tracks.

9.7 Flights and Data

The pipeline was evaluated on recordings of survey flights in Stonefish (Section 10), each holding the colour images, the images of the segmentation cameras and the true vehicle pose. Table 21 lists them. All were made in the reef of 70 corals. A recording can be replayed through the pipeline without the simulator, which is how every figure below was obtained, and which allows detectors and settings to be compared on exactly the same flight.

Flight	Area	Frames	Used for
Mound survey	16 m $\times$ 18 m over the reef mound	904 and 742	iteration 1
Full survey	north of $x = -8.5$ m, 805 m of track	2 521 and 2 440	iteration 2, training
Southern survey	strip south of $x = -8.5$ m	1 170 and 942	iteration 2, testing

Table 21. Recorded survey flights, all at 3 m above the seabed. Frame counts are for the downward and the forward camera.

9.8 Scoring

A map is scored against the positions of the corals in the scene file. Published landmarks and corals are paired one to one so that the sum of the horizontal distances of the pairs is least, and a pair counts when its distance is at most 1.5 m. Horizontal distance is used because a landmark lies at the middle of what was seen of a coral and the reference position is its base. From the pairs follow the number of landmarks that lie on a real coral, the number of corals mapped, the share of mapped corals with the correct family and the mean position error.

The number of corals mapped is stated with respect to the corals that the camera had a fair chance to see, and not with respect to all corals of the scene: a coral counts as having come into view when, in at least one frame of the flight, its position projected into the central 70 % of the image at a range below 8 m. The gate of 1.5 m is generous for small corals and should be kept in mind when the counts are read; the mean position errors are several times smaller.

9.9 Labelling Result

On the mound survey, 24 corals came into the central part of the downward image. The labelling procedure identified 22 of them, and for 21 of these the object number it chose received at least 99 % of the votes cast for that number. Two refinements were needed. A first version voted with a single point at the base of each coral and produced wrong labels, because seen from above the base point of one coral can lie under the canopy of its neighbour; voting with points on the surface removed this. Table corals then remained unlabelled, because the points of smaller corals hidden under their canopy vote for them as well and dilute their share; an object is now also assigned when a coral holds a clear share of its votes and the two are each other's first choice. Figure 19 shows labels produced in this way.

9.10 First Iteration

The first detector was trained on the mound survey alone: 3 972 training and 322 validation images after augmentation, a small YOLO segmentation model, 20 epochs at a resolution of 768 pixels, 35 minutes on a laptop graphics card with 6 GB of memory. The forward camera was level at that time. Table 22 gives the maps obtained from the downward camera.

The mapping stages worked: with ideal detections, 21 of 24 corals were mapped with a mean error of 0.19 m, and the learned detector gave almost the same map on the reef it was trained on. Two sensitivity checks on this flight support the method. With Gaussian noise of

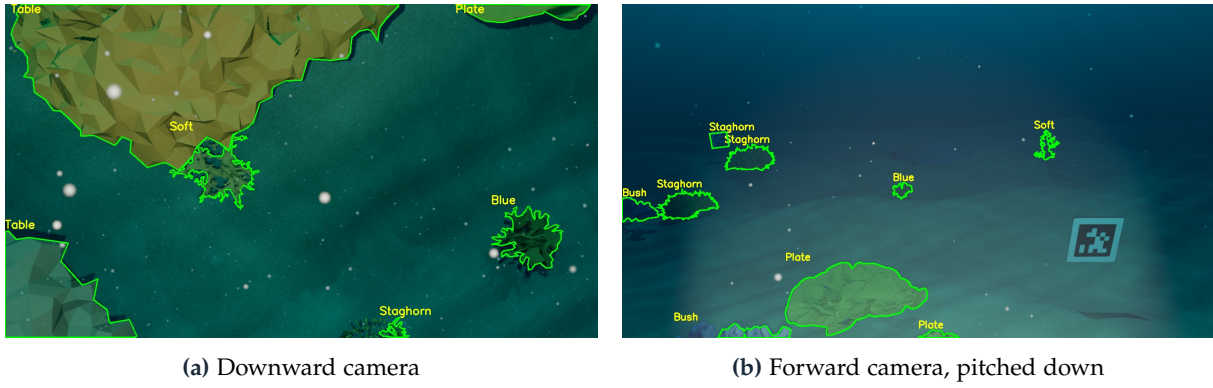


Figure 19. Automatically generated training labels from the full survey.

	Mound survey		Unseen corals	
	Oracle	Learned	Oracle	Learned
Landmarks published	24	24	9	7
Landmarks on a real coral	24	23	8	7
Corals in view that were mapped	21 of 24	20 of 24	7 of 11	7 of 11
Mapped corals with the correct class	21 of 21	19 of 20	7 of 7	3 of 7
Position error, mean	0.19 m	0.20 m	0.27 m	0.29 m

Table 22. First iteration: coral maps from the downward camera with the oracle and with the learned detector, true vehicle pose. The unseen corals were recorded on a short flight over an area outside the mound survey.

0.1 m added to the vehicle position, 19 of the 24 corals were mapped with a mean error of 0.24 m, and with the terrain model replaced by a flat seabed at 14 m, 20 of the 24 with a mean error of 0.20 m.

The iteration also showed two weaknesses. The detector had learned about thirty individual corals rather than nine families: on unseen corals it named the family correctly for three of seven. And the level forward camera was of no use for mapping, because from 3 m above the seabed it sees corals only from a distance and at a shallow angle: the same flight processed from the forward camera gave a single correct landmark.

9.11 Second Iteration

Both weaknesses were addressed.

Forward camera. The forward camera was pitched 20° downwards. From 3 m above the seabed its centre line then meets the seabed 8 m ahead and the lower edge of the image 3 m ahead, while a marker board at the depth of the vehicle remains inside the upper part of the image.

Training and test areas. The reef was divided. The full survey covers the 53 corals north of $x = -8.5$ m and supplied 6 388 training and 773 validation images from both cameras. Every frame that showed one of the 17 corals of the southern strip was left out, and the southern survey supplied the test data. The detector was trained as before. Table 23 gives its accuracy on the validation images. These show the same corals as the training images, so the figures measure how well the network learned the training area and are an upper bound on what to expect elsewhere. The Clathrata family is absent from the table because both of its corals lie in the southern strip.

Class	Images	Instances	Mask precision	Mask recall
Anemone	40	40	0.91	0.70
Antler	115	138	0.90	0.75
Blue	48	51	0.88	0.59
Bush	135	502	0.94	0.71
Plate	278	566	0.87	0.81
Soft	159	225	0.92	0.79
Staghorn	227	693	0.92	0.79
Table	94	160	1.00	0.88
All	773	2 375	0.92	0.75

Table 23. Second detector on the validation images of the training area. Mean average precision of the masks at an overlap threshold of 0.5 is 0.85 over all classes.

Results. Table 24 gives the maps of both surveys from both cameras.

Survey and camera	Detector	Landmarks (real)	Mapped	Class	Error
Training area, downward	oracle	51 (50)	50 of 53	48 of 50	0.20 m
Training area, downward	learned	51 (50)	50 of 53	48 of 50	0.20 m
Training area, forward	oracle	64 (53)	53 of 58	52 of 53	0.13 m
Training area, forward	learned	71 (56)	52 of 58	50 of 52	0.18 m
Southern strip, downward	oracle	13 (11)	11 of 13	11 of 11	0.23 m
Southern strip, downward	learned	13 (11)	11 of 13	9 of 11	0.24 m
Southern strip, forward	oracle	6 (3)	3 of 15	2 of 3	0.32 m
Southern strip, forward	learned	8 (6)	6 of 15	6 of 6	0.14 m

Table 24. Second iteration: coral maps, true vehicle pose, one flight each. The learned detector was trained on the training area only; the southern strip contains only corals that it had not seen. “Real” counts the landmarks that lie on a coral, “Mapped” the corals mapped out of those that came into view, “Class” the mapped corals with the correct family, and “Error” the mean position error.

Three conclusions follow. First, the pitched forward camera maps the training area as well as the downward camera does. Its maps contain duplicates, 64 landmarks for 53 corals, because a coral seen from two directions is not always merged into one landmark. Second, with corals from the whole northern reef in the training set, the detector carries over to corals it has not seen: from the downward camera it finds the same eleven of thirteen as the oracle and names the family correctly for nine of them, against three of seven in the first iteration. Third, the forward camera performs poorly on the southern strip with either detector, so the fault there does not lie in detection. On that flight the tracker started 1 265 tracks for about 35 corals, which means that tracks were broken continually, and the recording itself lacks 19 % of the flown track, because colour and segmentation frames failed to pair while the graphics card was occupied by another task. This has not been resolved.

The perception node was also run in the simulator with the learned detector on the forward camera, where it published tracked detections at 10 Hz. None of this has yet been repeated in the reef of 150 corals, 80 of which the detector has never seen.

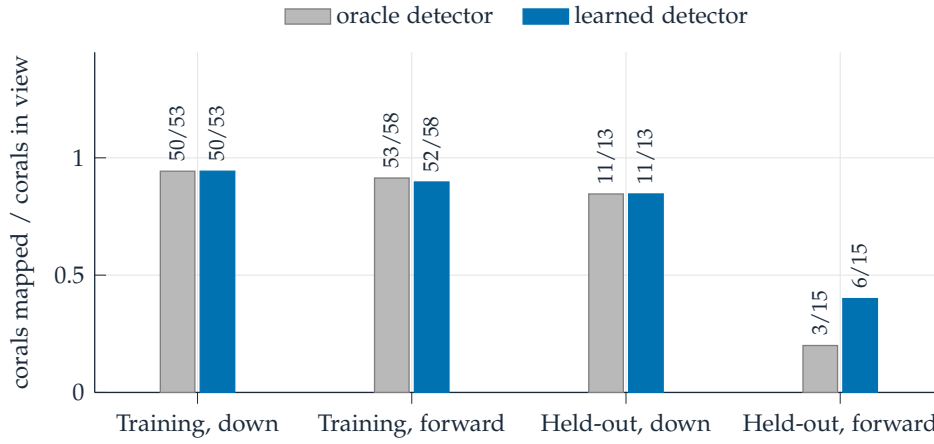


Figure 20. Share of the corals in view that were mapped, from Table 24. The learned detector matches the oracle in every configuration but one, and the poor result of the forward camera on the held-out strip occurs with both detectors.

10 Guidance

10.1 Waypoint Following

Autonomous flight was added without changing the controller. In position-hold mode the controller accepts velocity setpoints and holds position when the setpoint is zero. A separate guidance node turns a route into such setpoints. For a waypoint at (x_w, y_w, z_w) and a vehicle at (x, y, z) with heading ψ , the horizontal velocity setpoint is proportional to the position error, limited in magnitude to $v_{\max} = 0.3 \text{ m/s}$, and rotated into the body frame:

$$\begin{bmatrix} u_{\text{sp}} \\ v_{\text{sp}} \end{bmatrix} = \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} \text{sat}_{v_{\max}} \left(k_g \begin{bmatrix} x_w - x \\ y_w - y \end{bmatrix} \right). \quad (32)$$

The gain is $k_g = 0.6 \text{ s}^{-1}$, so the vehicle slows down over the last half metre before a waypoint. Heave and yaw rate are commanded in the same proportional, limited way, with limits of 0.3 m/s and 0.4 rad/s .

Heading. Unless a waypoint prescribes a heading, the vehicle turns to face the direction of travel, so that the forward camera looks where the vehicle is going. Within 1 m of the waypoint the heading is frozen, because the bearing to a point that is almost reached changes rapidly and would make the vehicle spin. While the heading error e_ψ is large, the horizontal velocity setpoint is scaled by

$$\lambda = \text{clip} \left(1 - \frac{|e_\psi| - 8^\circ}{25^\circ - 8^\circ}, 0.2, 1 \right), \quad (33)$$

which is one for an error below 8° and falls to its floor of 0.2 at about 22° . The vehicle thus turns almost on the spot at the end of a survey leg and only then accelerates, and the cameras do not sweep sideways across the scene, which would break the tracks of Section 9.

Arrival. A waypoint is reached when the vehicle is within 0.25 m of it horizontally and 0.3 m vertically, and within 8° of its heading if it prescribes one. The next waypoint then becomes the target. The node runs at 10 Hz, well above the 2 Hz below which the controller would treat its commands as stale.

Stopping. Every way in which the node can stop, including the end of the route, a state older than 0.5 s and a crash of the node itself, results in a zero or absent command, on which the controller holds position.

10.2 Survey Pattern

A survey is described by a rectangle, a spacing between legs and a height above the seabed. The generator lays out a back-and-forth pattern and cuts each leg into pieces of 1 m. The depth of the waypoint at (x, y) for a survey height H is

$$z_w(x, y) = \min_{|\Delta x|, |\Delta y| \leq 1 \text{ m}} z_s(x + \Delta x, y + \Delta y) - H, \quad (34)$$

where z_s is the depth of the seabed model, so that the vehicle follows the terrain and keeps its height above the shallowest ground in the neighbourhood of each waypoint, not above the ground directly beneath it. A mission file may also hold waypoints that are flown first, for example a climb and a transit to the survey area.

All surveys in this report were flown 3 m above the seabed, which clears the tallest coral (2.2 m). The spacing of the legs follows from the field of view. The downward camera has a horizontal field of view of 80° , and its image is 1280 pixels wide and 720 high with the top towards the bow, so at height H it covers a strip of width $2H \tan 40^\circ = 5.0$ m across the track and a length of 2.8 m along it. Legs 2.5 m apart therefore show every point of the seabed on two legs, from two sides, which is what the footprint method of Section 9.5 needs. At 0.3 m/s and the delivered frame rate of a few images per second, consecutive images overlap by more than 90 %.

10.3 Safety

The node has no knowledge of obstacles; the survey height is what keeps the vehicle clear of them. It aborts a mission when the reported state of the vehicle is not plausible (Section 11). It must not be run together with teleoperation, because both publish velocity commands and nothing yet arbitrates between them.

10.4 Result in Simulation

Three surveys were flown in the reef scene from start to finish without intervention, using the ground-truth state (Table 25). They provided the recordings used in Section 9.

Survey	Waypoints	Duration	Remark
Reef mound, 16 m $\times$ 18 m	136	about 40 min	depth tolerance 0.15 m
Full survey, north of $x = -8.5$ m	737	about 55 min	805 m of track
Southern strip	338	about 40 min	

Table 25. Unattended survey flights in Stonefish, 3 m above the seabed at up to 0.3 m/s.

On the first survey most of the time was spent on depth changes over the mound, because the vehicle waited at every waypoint until its depth was within 0.15 m. The tolerance was then relaxed to 0.3 m and the vertical speed limit raised from 0.2 to 0.3 m/s.

11 Verification and Faults Found

11.1 Approach

The software is tested at three levels, and each fault described in this report was found at the lowest level able to show it.

Unit tests. The arithmetic of every package is kept in modules that do not import ROS 2, and the nodes are thin wrappers around them. These modules are covered by 156 unit tests that run in under a minute without the simulator (Table 26). They include tests on rendered marker images (Section 8) and a synthetic survey: a simulated camera flies over twelve corals drawn as spheres on a flat seabed, and the frames pass through the whole perception pipeline. All twelve are mapped with a mean position error of 0.02 m, and with 0.05 m of noise on the vehicle position and a seabed model that is 0.5 m too deep the mean error is 0.09 m. Such a test checks the geometry of the pipeline, not its behaviour on real shapes.

Package	Tests	Subject
Control	51	allocation matrix and saturation, frame conversions and attitude signs, PID with clamping
Marker localization	51	geometry, marker map, detection on rendered images, pose and covariance, gating, scene generation and start clearance
Perception	45	labelling, tracker, ranging, map, scoring, augmentation, the synthetic survey
Guidance	9	velocity command, arrival, survey pattern, state plausibility

Table 26. Unit tests that run without ROS 2 or the simulator. The teleoperation package has further tests that need ROS 2 and run inside the container.

Node tests without the simulator. The nodes are started from their launch files and supplied with rendered images and synthetic sensor data over the ROS 2 topics they use in operation. This checks message types, parameters and frames, and it exposed two faults of the localization before any simulator run (Section 8).

Flights in the simulator. Survey flights are recorded and replayed (Section 9), and every start is checked from the odometry before a mission or a recording begins.

11.2 A Simulator Fault: Uninitialised Thruster Setpoints

On some starts the vehicle did not respond, the simulator reported speeds above a thousand metres per second, and the vehicle either stayed frozen at its start position or left the scene. The same state had occurred earlier during teleoperation, after contact with corals and, at other times, without any contact.

Four explanations were considered and set aside before the cause was found (Table 27); three were refuted by evidence and one could not be tested. The first is recorded in some detail because the way it failed is instructive. The failures first appeared after the start position had been moved to the seabed, and an estimate based on the bounding circle of each coral placed that position inside a table coral. Moving the start position was followed by a series of clean starts, which appeared to confirm the explanation. It did not: measured against the actual mesh, the first position had been 0.68 m away from the coral, and the failures returned a day later at the new position, in 17 of 20 starts. A fault that occurs at random will sooner or later appear to respond to any change that is made.

The cause was found by attaching a monitor before the first physics step. In every failed start the first odometry sample already showed each of the six velocity coordinates at ± 1000 , which is the limit that the physics engine imposes, while every thruster command sent up to that moment had been zero. The outcome of a start therefore did not depend on anything the control software did. A result that varies between identical runs of a deterministic program points to memory that is read before it is written, and this was tested by filling freshly allocated memory deliberately (Table 28). With a junk pattern every start failed, and with another fill

Explanation	Evidence against it
The vehicle starts inside a coral	the hull was 0.68 m clear of the nearest mesh; failures persisted at a position with more clearance
The graphics card is overloaded by a training job	failures persisted with the training job paused
A race between threads of the simulator	could not be tested as intended: the setting used to restrict the thread count does not apply to the simulator's own thread pool
The control software commands a large thrust	every thruster command sent before the first faulty sample was zero

Table 27. Explanations of the start-up fault that were examined and set aside.

the failures changed in kind, from saturated velocities to values that are not numbers. The outcome followed the fill. The monitor used in the first experiments had itself to be corrected: it compared the speed with a threshold, and a state that is not a number fails every comparison and was reported as sane.

Freshly allocated memory contains	Clean starts before	Clean starts after
Whatever the process left there (normal operation)	3 of 20	10 of 10
A junk byte pattern	0 of 6	5 of 5
Zeros, with recycled blocks set to all ones	2 of 6	4 of 4

Table 28. Simulator starts with a sane vehicle state, before and after the correction, for three ways of filling newly allocated memory (the `MALLOC_PERTURB_` facility of the C library). Before the correction the failed starts in the third row reported non-numeric values rather than saturated velocities.

The source of Stonefish shows the reason. The constructor of the simplified thruster model sets its output force and torque to zero but does not initialise the two setpoints from which they are copied at every physics step. The ROS 2 interface writes the setpoints only when a thruster command has been received, and it can receive none until after the physics thread has started. For the first step or two each of the eight thrusters therefore applied whatever number its memory happened to hold, which is usually astronomically large. The interface now sets the setpoints of every such thruster to zero when the scene is built, before the simulation starts. The fault is present in the latest upstream version of the simulator.

11.3 Contact with Obstacles

Contact was then tested on its own. With the correction in place, the vehicle was driven into a plate coral, a branching coral, a table coral, the seabed and a marker board at 0.3 to 1.0 m/s, lowered onto two corals from above, and ground against two corals for 45 s in manual mode with full thrust and alternating roll and yaw torque. In every one of these tests it came to rest against the obstacle and its state remained sane. The only contact fault that could be produced is a start in which the hull already overlaps a collision mesh, which throws the vehicle out of the scene within a tenth of a second (three starts of three). The scene generator now measures the distance from the start position to the nearest vertex of every collision mesh and refuses a position closer than 1 m.

11.4 Guards

Two guards were added so that a fault of this kind cannot pass unnoticed again. The guidance node aborts a mission, commands zero and reports the reason when the reported state of the vehicle ceases to be plausible, that is, when it is not finite, faster than 5 m/s or outside the operating volume. The recorder keeps no frames from such a state. The flight during which the fault was last seen had gone on recording for several minutes while the vehicle was hundreds of kilometres from the reef.

12 Discussion and Limitations

12.1 Validity of the Results

Four limitations apply to every result from Section 5 onwards and determine how far the figures of this report may be generalised.

Simulation only. No experiment was carried out on the Version 3 hardware. The simulated thrusters are ideal force sources, the water is still, and the camera is a pinhole without distortion that sees water of one optical type. The results show that the software is correct with respect to its own model and establish a method; they do not predict the performance of the vehicle.

Ground-truth state. The vehicle state used by the controller, the guidance and the perception pipeline in all reported flights is the ground truth of the simulator. The marker localization was validated on its own against that ground truth and has not been used to close a loop. The sensitivity checks of Section 9, with 0.1 m of noise on the vehicle position, indicate that the mapping tolerates a realistic position error, but the complete chain on estimated state has not been flown.

Single runs. Every table of measurements reports single runs. Differences between configurations of a few per cent, or of one or two corals, are within what a repetition could change, and the conclusions drawn in the text rest only on differences that are large: a factor of seven in inertia, a factor of two in localization error between viewing geometries, three corals of fifteen against fifty of fifty-three.

One reef. The detector was trained and evaluated in one simulated reef of 70 corals. The held-out corals are instances that the detector has not seen, at their own positions, scales and headings; whether their models also occur in the training area was not controlled, so the result measures generalisation to unseen instances and not necessarily to unseen shapes. One family is absent from the training data. The results have not been repeated in the reef of 150 corals.

12.2 Known Issues

State estimation is not closed. The controller runs on ground truth. On the estimate of Section 7 it holds depth and attitude, but the heading drifts with the inertial sensor at about 3° per minute, and position hold wanders by 0.19 m and exceeds the roll target. The filter takes its horizontal position from a noisy copy of the ground truth, has no measurement of velocity and no model of the vehicle, and does not reject outliers. The marker localization can replace the position input, but markers are visible only near the boards, and between fixes the horizontal estimate degrades within seconds.

The controller is not robust. Roll damping is low and the gain of the roll loop is bounded by a loop delay of about 90 ms, part of which is rendering load and will differ on the vehicle. The axes are tuned as if independent, although sway couples into roll. Currents,

thruster dynamics and sensor noise have not been tested, and a saturated allocator reduces attitude torque together with thrust.

The simulated plant is unverified. The added mass that the simulator derives is an order of magnitude above the dry mass, and neither it nor the drag has been compared with the hardware. Gains found in simulation are a starting point for the pool and no more.

Simulation artefacts. A vehicle that starts with its hull overlapping a collision mesh is thrown out of the scene by the physics engine. The scene generator guards against this for the start position, but not for objects added to a scene by hand. The thruster fault of Section 11.2 is corrected in the interface package of this project and remains in the simulator itself.

Rendering rate. Near the reef the simulator delivers about five camera images per second in total, which is slow for piloting and for a perception node that runs live.

Corals larger than the field of view. From 3 m above the seabed the downward camera never sees a table coral whole. Such corals are placed by the union of their partial outlines, which is less accurate than triangulation.

Forward-camera mapping is fragile. It works on the survey of the training area and fails on the southern strip, where tracks break continually (Section 9.11). Its maps also contain duplicate landmarks.

Marker accuracy depends on view and range. With the pitched camera the mean error is 0.05 m when the vehicle faces the board and 0.10 m when it is parallel to it, and neither figure has been separated from the timing of the comparison. Both were measured at 4 to 5 m. From a single board the position across the line of sight is not constrained by the attitude prior and its uncertainty grows with the cube of the range (Section 8.5); nothing has been measured beyond 5 m.

Seabed model. The first range estimate assumes a model of the seabed. In simulation the true terrain is used. At sea it would have to come from an altimeter or be assumed flat, which was tested only as a sensitivity check.

12.3 Future Work

The items are ordered by their dependence on one another; the first is the precondition of most of the others.

1. Fuse the marker fixes, including their heading, in the state estimator with an innovation gate; add a velocity source or a prediction from the commanded wrench; retune the horizontal loops for a noisy state; and repeat the hold and survey tests on the estimated state.
2. Identify added mass and drag on the hardware, add thruster dynamics and a current to the simulation, and add a disturbance observer to the controller.
3. Replace the scaled pseudo-inverse by a constrained allocation that respects the thruster limits and gives attitude priority.
4. Place boards so that two are in view where position matters, or add a heading reference, to remove the weak direction of single-marker fixes.
5. Add a command arbiter so that teleoperation always overrides autonomous commands, and failsafes for a lost or diverged state estimate.

6. Add a mission layer above the guidance, for example a state machine or behaviour tree that sequences search, approach, inspection and return.
7. Record and train in the reef of 150 corals, evaluate the detector on a reef with different coral models, and retrain it with footage from the pool and from open water.
8. Find why forward-camera tracks break on the southern strip, and merge duplicate landmarks.
9. Repeat the principal measurements several times, so that they can be reported with their spread.
10. Add the competition objects (gates, buoys, markers on the floor) as detector classes.
11. Map the thruster forces to motor commands and bring the control stack onto the on-board computer of Version 3.

13 Conclusion

The project began with a vehicle that leaked and could not be simulated. It now has a sealing method that holds, a simulation that the whole team can run, closed-loop control in six degrees of freedom, and the first components of autonomy: localization from markers, recognition and mapping of corals with a single camera, and unattended survey flight. In simulation and on ground-truth state, the controller holds depth to 0.4 cm and tracks its commanded rates within 10%; the marker localization reaches a mean position error of 0.05 m at 4 m; and the downward camera maps 50 of 53 corals in view with a mean error of 0.20 m, and eleven of thirteen corals that the detector had not seen.

Two lessons recur throughout the work. The first is that faults were found by measurement and not by inspection. The grey corals were a property of light in water and not of the models. The vehicle that was thrown out of the scene had touched nothing; its thrusters were applying setpoints that had never been set. The localization that passed every unit test rejected good fixes as soon as it was driven with realistic attitude errors. And the controller gains derived from an assumed mass were too weak by a factor of three or more, because the simulated vehicle responds as if it had twelve times its dry mass. The second lesson is that an estimate is only useful with an honest uncertainty. The marker fixes and the coral positions both carry covariances that were checked against the actual scatter, and it is these, more than the estimates themselves, that will allow a state estimator and a mission planner to use them safely.

The step that remains before the vehicle can be called autonomous is to close the loop on estimated instead of true state, first in simulation and then in the pool.

Acknowledgments

ROVIO is the work of a team, and this report describes one part of it.

In the simulation group, I thank Kuhan Anandh, who made the first Stonefish implementation in our team, a floating cube, and set up the repository from which this work started, and Yagishvar, who advised on the teleoperation. I thank Aditya Ganta for his help with the conversion of the model from the east-north-up to the north-east-down frame.

The vehicle was designed by Aditya Ganta, Arjun and Sreedath, and Arjun was responsible for its water sealing. Saiprakash carried out the procurement and the manufacturing in Coimbatore, and Saiprakash and Srijan designed the printed circuit board. Kishore, Kazim, Sree Vidya and Nandgovind assisted in water sealing and assembly. I thank them, and Krithyaa, Vignesh and all other members of the team who took part.

I thank the Robotics and Machine Intelligence club for funding the project. The simulation rests on the Stonefish simulator and its ROS 2 interface by Patryk Cieślak.

References

- [1] Musa Morena Marcusso Manhães, Sebastian A. Scherer, Martin Voss, Luiz Ricardo Douat, and Thomas Rauschenbach. UUV simulator: A Gazebo-based package for underwater intervention and multi-robot simulation. In *OCEANS 2016 MTS/IEEE Monterey*, pages 1–8, 2016. doi: 10.1109/OCEANS.2016.7761080.
- [2] Mabel M. Zhang, Woen-Sug Choi, Jessica Herman, Duane Davis, Carson Vogt, Michael McCarrin, Yadunund Vijay, Dharini Dutia, William Lew, Steven Peters, and Brian Bingham. DAVE aquatic virtual environment: Toward a general underwater robotics simulator. In *2022 IEEE/OES Autonomous Underwater Vehicles Symposium (AUV)*, pages 1–8, 2022. doi: 10.1109/AUV53081.2022.9965808.
- [3] Easton Potokar, Spencer Ashford, Michael Kaess, and Joshua G. Mangelson. HoloOcean: An underwater robotics simulator. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 3040–3046, 2022. doi: 10.1109/ICRA46639.2022.9812353.
- [4] Patryk Cieślak. Stonefish: An advanced open-source simulation tool designed for marine robotics, with a ROS interface. In *OCEANS 2019 - Marseille*, pages 1–6, 2019. doi: 10.1109/OCEANSE.2019.8867434.
- [5] Derya Akkaynak and Tali Treibitz. A revised underwater image formation model. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6723–6732, 2018. doi: 10.1109/CVPR.2018.00703.
- [6] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), 2022. doi: 10.1126/scirobotics.abm6074.
- [7] Evan Palmer, Christopher Holm, and Geoffrey Hollinger. Angler: An autonomy framework for intervention tasks with lightweight underwater vehicle manipulator systems. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6126–6132, 2024. doi: 10.1109/ICRA57147.2024.10610184.
- [8] Thor I. Fossen. *Handbook of Marine Craft Hydrodynamics and Motion Control*. John Wiley & Sons, Chichester, 2011.
- [9] Gianluca Antonelli. *Underwater Robots*. Springer, Cham, 4 edition, 2018.
- [10] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6235–6240, 2015. doi: 10.1109/ICRA.2015.7140074.
- [11] Karl Johan Åström and Tore Hägglund. *Advanced PID Control*. ISA - The Instrumentation, Systems, and Automation Society, Research Triangle Park, NC, 2006.
- [12] Wen-Hua Chen, Jun Yang, Lei Guo, and Shihua Li. Disturbance-observer-based control and related methods: An overview. *IEEE Transactions on Industrial Electronics*, 63(2):1083–1095, 2016. doi: 10.1109/TIE.2015.2478397.

-
- [13] Tor A. Johansen and Thor I. Fossen. Control allocation: A survey. *Automatica*, 49(5): 1087–1103, 2013. doi: 10.1016/j.automatica.2013.01.035.
 - [14] Thomas Moore and Daniel Stouch. A generalized extended Kalman filter implementation for the Robot Operating System. In *Intelligent Autonomous Systems 13, Advances in Intelligent Systems and Computing*, pages 335–348. Springer, Cham, 2016. doi: 10.1007/978-3-319-08338-4_25.
 - [15] Edwin Olson. AprilTag: A robust and flexible visual fiducial system. In *2011 IEEE International Conference on Robotics and Automation*, pages 3400–3407, 2011. doi: 10.1109/ICRA.2011.5979561.
 - [16] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez. Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47(6):2280–2292, 2014. doi: 10.1016/j.patcog.2014.01.005.
 - [17] Diego Brito dos Santos Cesar, Christopher Gaudig, Martin Fritsche, Marco A. dos Reis, and Frank Kirchner. An evaluation of artificial fiducial markers in underwater environments. In *OCEANS 2015 - Genova*, pages 1–6, 2015. doi: 10.1109/OCEANS-Genova.2015.7271491.
 - [18] Gerald Schweighofer and Axel Pinz. Robust pose estimation from a planar target. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(12):2024–2030, 2006. doi: 10.1109/TPAMI.2006.252.
 - [19] Toby Collins and Adrien Bartoli. Infinitesimal plane-based pose estimation. *International Journal of Computer Vision*, 109(3):252–286, 2014. doi: 10.1007/s11263-014-0725-5.
 - [20] Kenneth Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2(2):164–168, 1944. doi: 10.1090/qam/10666.
 - [21] Donald W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963. doi: 10.1137/0111030.
 - [22] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016. doi: 10.1109/CVPR.2016.91.
 - [23] Glenn Jocher, Jing Qiu, and Ayush Chaurasia. Ultralytics YOLO. <https://github.com/ultralytics/ultralytics>, 2023.
 - [24] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. doi: 10.1002/nav.3800020109.
 - [25] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016. doi: 10.1109/ICIP.2016.7533003.
 - [26] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. ByteTrack: Multi-object tracking by associating every detection box. In *Computer Vision - ECCV 2022, Lecture Notes in Computer Science*, pages 1–21. Springer, 2022. doi: 10.1007/978-3-031-20047-2_1.
 - [27] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, Cambridge, 2 edition, 2004.

-
- [28] Javier Civera, Andrew J. Davison, and J. M. Martínez Montiel. Inverse depth parametrization for monocular SLAM. *IEEE Transactions on Robotics*, 24(5):932–945, 2008. doi: 10.1109/TRO.2008.2003276.
- [29] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017. doi: 10.1109/IROS.2017.8202133.
- [30] Gary Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 2000.

Appendices

A Chronology of the Project

Table 29 summarises the work in the order in which it was done. The dates are those of the commits in the project repository, which holds about eighty commits between December 2025 and October 2026. The sealing work of Section 4 preceded the repository and is not recorded in it.

Period	Work
December 2025	First Stonefish implementation in the team, by Kuhan Anandh: a floating cube, the Stonefish ROS 2 interface and installation notes, in a new repository.
December 2025 to January 2026	Vehicle model brought into Stonefish: a first working scene, correction of frames and inertia with the help of Aditya Ganta, re-orientation of the thruster meshes in Blender.
February 2026	Teleoperation package with thruster allocation, with advice from Yagishvar; camera added to the vehicle; correction of thruster directions and of a residual torque; a single launch file for the whole system.
March to June 2026	Consolidation after the institute’s technical festival, Pragyan.
18 September 2026	Docker environment; rewritten documentation; control package with a cascaded controller for six degrees of freedom, keyboard teleoperation and a tuning guide.
19 to 21 September 2026	Coral reef generator, kept in a repository of its own; reef scene and vehicle lights; gamepad teleoperation routed through the controller.
23 September 2026	Control software moved to the marine frame conventions; controller tuned by step tests; work on the state estimator begun on a separate branch.
4 October 2026	Low-polygon coral meshes; marker localization; perception pipeline.
5 October 2026	Downward camera, start position near the seabed and coral colours that survive absorption; guidance package and survey flights; detector training; forward camera pitched down; thruster fault found and corrected; reef enlarged to 150 corals.

Table 29. Chronology of the project, from the commit history of the repository.

One observation from this history shaped the later work. The first months were spent almost entirely on getting a correct model into the simulator, and the errors made there (frames, thruster directions, torque) were all found by driving the vehicle and seeing it misbehave. From September onwards each piece of software was therefore written with tests that do not need the simulator, so that a fault in the geometry or the arithmetic is found before a flight is attempted (Section 11).

B Deliverables

The following exist and run at the time of writing.

1. **A sealing method** for the enclosure, tested in hardware and used on Version 3 (Section 4).
2. **A simulation of the Version 3 vehicle** in Stonefish with eight thrusters, an inertial sensor, a pressure sensor, forward and downward cameras with lamps, and a generated reef of 150 corals (Section 5).
3. **A reef generator**, in a repository of its own, that builds the reef scene from a layout file (Section 5.5).

4. **A containerised environment** that builds and runs the whole system with one command on any Linux machine with a supported graphics card.
5. **A control stack** consisting of a thruster allocator and a cascaded controller with four modes, with gamepad and keyboard teleoperation, tools for step, hold and sign tests, and a record of the plant measurements and the tuning (Section 6).
6. **A state estimator in progress**, on a separate branch: an extended Kalman filter on the inertial and pressure sensors with the adapter that feeds it. The controller holds depth and attitude on its output; heading and horizontal position are not yet adequate (Section 7). It is not used in the results of the other sections.
7. **Marker-based localization** that publishes the vehicle pose with a covariance for fusion in a state estimator (Section 8).
8. **A coral perception pipeline** with two detectors, a tracker, range estimation from a single camera and a persistent map, together with tools that record labelled data from the simulator, train the detector and score a map against the scene (Section 9).
9. **A guidance package** that flies waypoint routes and terrain-following surveys (Section 10).
10. **A correction to the simulator interface** that removes a fault which corrupted the vehicle state at start (Section 11.2).
11. **A test suite** of 156 unit tests (Section 11).

The control, localization, perception and guidance functions are separate ROS 2 packages that communicate through topics, so that each can be replaced or developed on its own.